
Cloud Computing Project

Design and Evaluation of an Intelligent, Scalable, and Cost-Aware Cloud Data Pipeline for Real-Time and Batch Analytics

Group 7

Krishna Agrawal

2025H1030050H | h20250050@hyderabad.bits-pilani.ac.in

Patel Harshkumar J

2025H1030179H | h20250179@hyderabad.bits-pilani.ac.in

Fathima Fiza B

2025H1030057H | h20250057@hyderabad.bits-pilani.ac.in

Khushi Dusad

2025H1030052H | h20250052@hyderabad.bits-pilani.ac.in



Department of Computer Science
BITS Pilani, Hyderabad Campus

Contents

1	Abstract	5
2	Introduction	6
3	Problem Statement	7
4	Input Event Model	7
4.1	Schema Organization	
4.1.1	Event Metadata	
4.1.2	User Profile Attributes	
4.1.3	Contextual and Device Information	
4.1.4	Session and Course Context	
4.1.5	Interaction-Level Details	
4.2	Event Types and Lifecycle	
5	Pipeline 1: Serverless Architecture (Baseline)	9
5.1	Architecture Overview	
5.2	Data Generation Layer	
5.3	Data Ingestion Layer	
5.4	Processing Layer	
5.5	Intelligence Layer	
5.6	Storage and Metadata Layer	
5.7	Query Layer	

5.8	Visualization Layer	
6	Challenges Faced in the Serverless Architecture	20
7	Limitation of Serverless Architecture	21
8	Motivation for Alternative Architecture	21
9	Pipeline 2: Distributed Streaming Architecture (Optimized)	22
9.1	Architecture Overview	
9.2	Data Generation Layer	
9.3	Data Ingestion Layer	
9.4	Processing Layer	
9.5	Intelligence Layer	
9.6	Scalable Storage Layer	
9.7	Batch Processing Layer	
9.8	Monitoring and Metrics Layer	
9.9	Visualization Layer	
9.10	Spark Streaming Logs and System Behavior	
9.11	Checkpoint-Based Failure Recovery	
9.12	Performance and Cost Benchmarking	
9.13	Experimental Evaluation	
9.14	Research Analysis	

10 Comparison of the Two Pipelines	60
11 Conclusion	64

Abstract

This project presents the design and evaluation of a cloud-native data pipeline for an EdTech analytics use case involving continuous streams of student activity events. The system is developed in two stages. In the first stage, a serverless AWS pipeline is implemented using Lambda for real-time processing, S3 for raw storage, AWS Glue for batch transformation, Athena for querying, and QuickSight for dashboarding. This baseline pipeline demonstrates the feasibility of building a unified real-time and batch analytics workflow on managed cloud services. However, during analysis, several limitations of the serverless design become apparent, particularly under sustained or bursty streaming workloads, including higher invocation-driven cost, stateless execution overhead, limited suitability for continuous processing, and weaker control over resource allocation.

To address these limitations, a second pipeline is implemented using a distributed streaming architecture built on Kafka, Spark Structured Streaming, Redis, S3, DynamoDB, Prometheus, Grafana, and Kubernetes. In this design, events are generated on one EC2 instance and processed continuously on another, with Kafka providing buffered ingestion, Spark performing real-time processing and intelligence, and Redis serving low-latency metrics. Storage operations are decoupled from processing and handled by lightweight Kubernetes-based worker pods, which write data to S3 and DynamoDB. These pods are dynamically scaled using KEDA based on Kafka consumer lag, while EC2 Auto Scaling ensures sufficient infrastructure capacity under high load.

Two rule-based intelligence components are integrated within the Spark processing layer: AdaptiveBatchOptimizer for cost-aware S3 writes and SurgeDetector for SLA-aware priority routing during spikes. The final evaluation is based on controlled experiments measuring cost, latency, surge handling, scaling behavior, and failure recovery. Results show that the optimized pipeline reduces S3 API costs by 87.6%, detects and handles surges within one micro-batch, completes the batch reporting job in 118 seconds on 267,759 events, and recovers from failures within approximately 5 seconds. These results demonstrate that a carefully designed distributed streaming pipeline can provide better control, efficiency, scalability, and resilience for continuous high-volume workloads.

Introduction

Modern digital platforms continuously generate large volumes of data that must be collected, processed, stored, and analyzed in near real time. In domains such as online learning, events such as student logins, class joins, quiz submissions, and session terminations can provide valuable information about engagement, performance, and system behavior. However, these event streams are often high velocity, bursty, and operationally complex. Traditional ETL approaches are not sufficient for such workloads, because they do not support continuous ingestion and real-time visibility effectively. The course project specifically frames this as a cloud data pipeline problem involving real-time and batch processing, cost–latency trade-offs, adaptive routing, observability, and fault handling.

The goal of this project is to design and evaluate an intelligent, scalable, and cost-aware cloud data pipeline for EdTech activity analytics. The selected use case is an online learning platform in which synthetic student activity events are generated continuously and processed for both real-time monitoring and historical analysis. The project follows the instructor’s recommended execution logic of first building a simple working pipeline and then evaluating and improving it through controlled experiments.

To achieve this, the project is developed in two stages. The first stage implements a baseline serverless pipeline on AWS using Lambda, S3, Glue, Athena, QuickSight, and DynamoDB. This architecture is useful for demonstrating the core cloud workflow and validating end-to-end data movement from ingestion to analytics.

In the second stage, an improved pipeline is implemented using a distributed streaming architecture deployed on EC2 instances, combined with a Kubernetes-based scaling layer. The system integrates Kafka for ingestion, Spark Structured Streaming for real-time processing, Redis for low-latency analytics, S3 and DynamoDB for storage, and Prometheus with Grafana for monitoring. A key enhancement in this design is the separation of processing and storage responsibilities, where scalable Kubernetes-based storage workers handle high-throughput writes independently of the Spark processing layer.

This second design also introduces two intelligence mechanisms, AdaptiveBatchOptimizer and SurgeDetector, to improve cost efficiency and SLA behavior under variable workloads. In addition, KEDA-based scaling and EC2 Auto Scaling enable the system to dynamically adapt to workload changes. The report therefore compares a simple baseline against an optimized streaming system, aligning with the academic objective of evaluating baseline and improved architectures under workload-driven conditions.

Problem Statement

The objective of this project is to design a scalable cloud-based data pipeline capable of processing both real-time and batch data streams while maintaining low latency, high throughput, and cost efficiency. The system should handle dynamic workloads, including burst traffic, and provide reliable analytics outputs.

The project specifically focuses on evaluating different architectural approaches to determine their suitability for continuous streaming applications. Key evaluation criteria include latency, throughput, cost efficiency, fault tolerance, and operational complexity.

Input Event Model

The system adopts a unified event-driven data model to represent user interactions within an EdTech platform. Each interaction is captured as a structured JSON object, enabling seamless integration across ingestion, processing, storage, and analytics components.

Schema Organization

The event schema is logically divided into five categories to improve clarity, maintainability, and analytical usability.

4.1.1 Event Metadata

These fields uniquely identify and describe the nature of the event.

Field	Description
event_id	Globally unique identifier for the event
event_type	Type of user action (e.g., login, class join, session start)
timestamp	Time of event generation in ISO 8601 format
schema_version	Version of the schema for compatibility

4.1.2 User Profile Attributes

These attributes describe the characteristics of the user generating the event.

Field	Description
user_id	Unique user identifier
user_role	Role of user (student/instructor)
subscription_type	Free or premium subscription user
age_group	Age segment
user_level	Skill level (beginner/intermediate/advanced)

These attributes remain consistent across events for a given user.

4.1.3 Contextual and Device Information

These fields provide environmental context for each interaction.

Field	Description
region	User's geographic region
country	Country of origin
timezone	Timezone of the user
device	Device type (mobile/desktop/tablet) os
	Operating system
app_version	Version of the application

These attributes support segmentation analysis and system-level insights.

4.1.4 Session and Course Context

These fields associate user actions with specific sessions and classes.

This group enables tracking of user participation across courses.

4.1.5 Interaction-Level Details

These attributes capture the specific nature of user engagement. These fields are critical for fine-grained engagement analytics.

Field	Description
session id	Identifier for a user session
class id	Identifier for the class
course name	Name of the course
course category	Subject domain
difficulty level	Course difficulty classification
instructor id	Identifier of the instructor

Field	Description
session type	Type of session (live/recorded)
interaction type	Activity type (video, quiz, poll, etc.)
content id	Identifier for content being accessed
content duration	Duration of interaction in seconds

Event Types and Lifecycle

The system models user behavior through a set of discrete event types:

- **login / logout**: user authentication lifecycle
- **class join / class leave**: class participation lifecycle
- **session start / session end**: content interaction lifecycle

These events follow a logical progression:

login → class join → session start → session end → class leave → logout

Pipeline 1: Serverless Architecture (Baseline)

The first pipeline is implemented as a serverless AWS-based analytics architecture for processing EdTech user activity events in near real time. The purpose of this pipeline is to ingest interaction events, process them immediately for operational insights, preserve the raw data for later analysis, and expose both historical and live views through SQL queries and dashboards. The architecture combines Amazon Kinesis for event ingestion, AWS Lambda for stream processing, Amazon S3 for raw data storage, Amazon DynamoDB for real-time state management, AWS Glue for schema discovery, Amazon Athena for SQL-based querying,

and Amazon QuickSight for dashboard visualization. This design matches the project goal of supporting both real-time and batch analytics in a cloud-native manner.

Architecture Overview

Pipeline 1 follows two parallel analytics paths after Lambda processing. In the first path, raw JSON events are stored in Amazon S3 and later queried through AWS Glue and Amazon Athena for historical analysis. In the second path, Lambda writes processed operational state into DynamoDB tables, which are then exposed to Athena through Athena Federated Query and converted into views for dashboarding. As a result, the pipeline supports both batch-style historical analytics and near-real-time operational analytics within the same overall architecture.

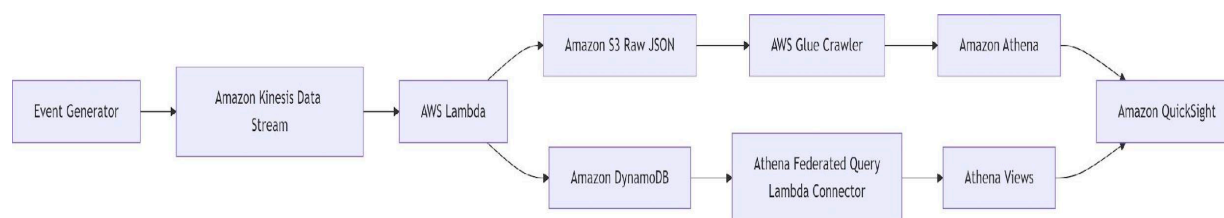


Figure 1: Pipeline 1 Architecture

Data Generation Layer

The system follows a structured event-driven model in which user interactions are generated as JSON events by a custom event generator running on a local machine. This generator simulates realistic EdTech platform behavior and provides configurable controls such as event rate (events per second), number of active users, and workload patterns (e.g., continuous or spike modes). These controls allow the system to emulate different traffic conditions, ranging from steady usage to bursty real-world scenarios.

By enabling precise control over input characteristics, the generator ensures repeatable experiments and flexible testing of system performance under varying loads. The generated events follow valid interaction sequences, maintaining consistency in session and class lifecycles. This controlled yet realistic data generation forms a reliable foundation for evaluating the ingestion, processing, and analytics capabilities of the overall system.

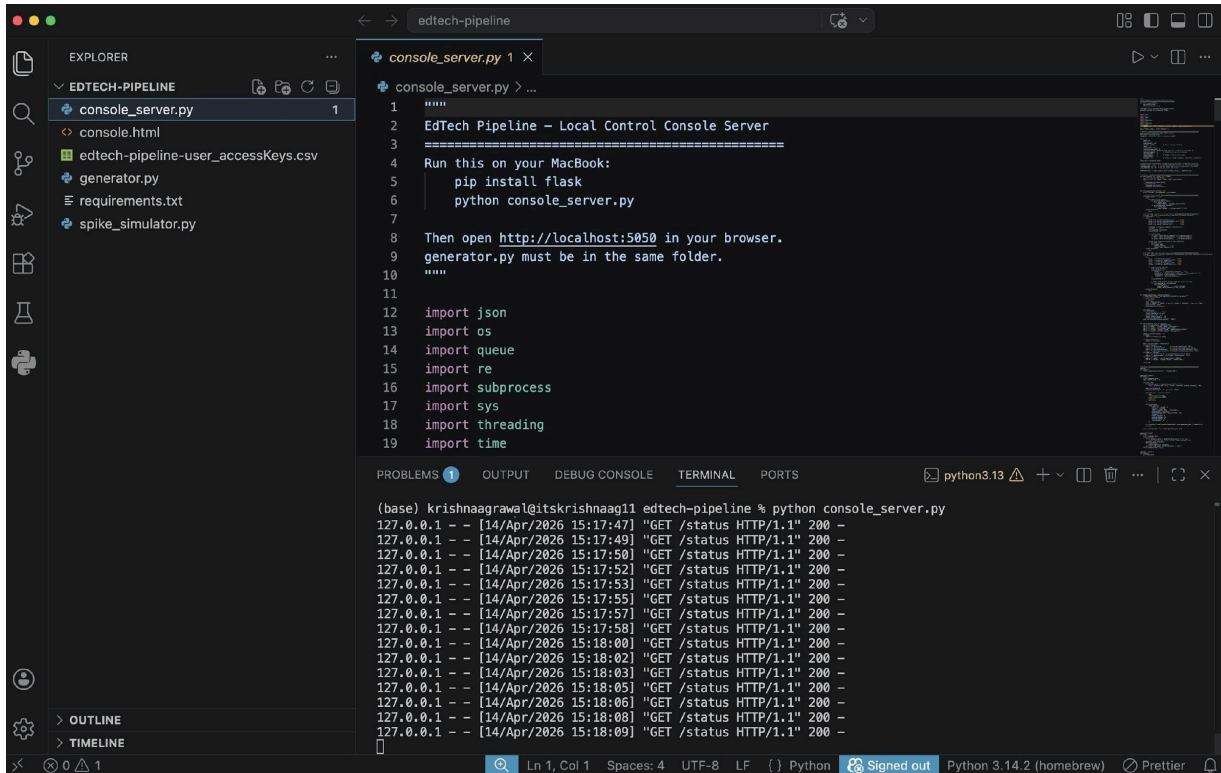


Figure 2: Running the generator console on local machine

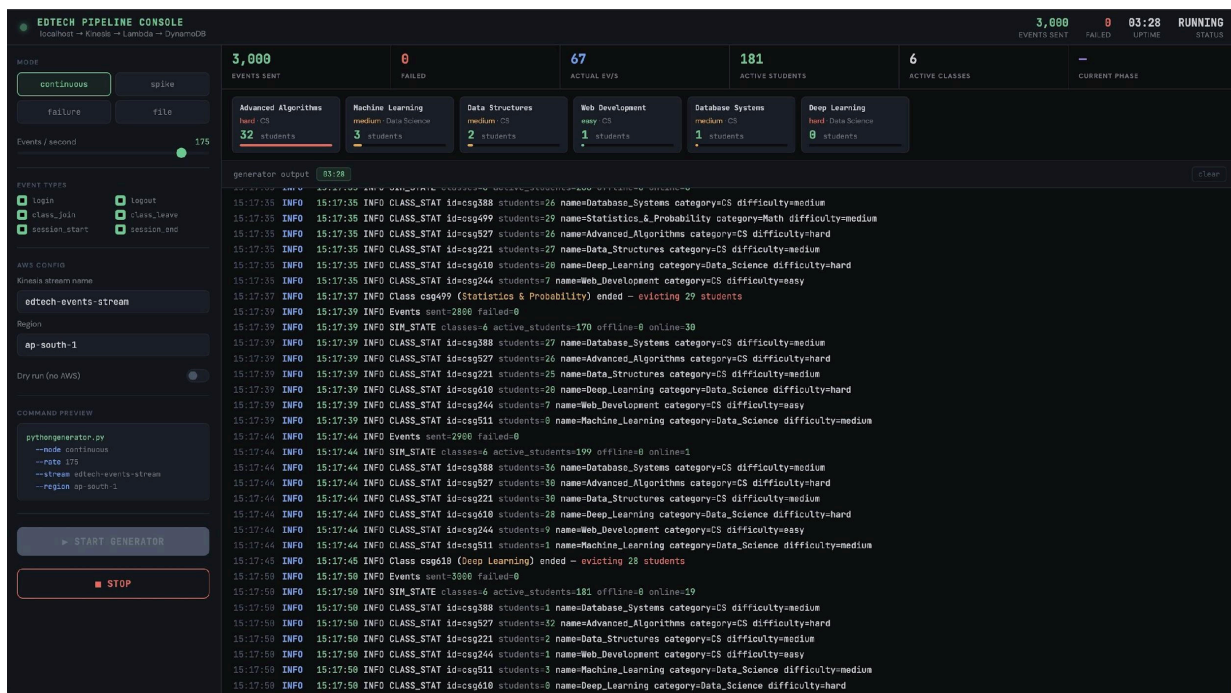


Figure 3: Generator Console

Data Ingestion Layer

The ingestion layer uses Amazon Kinesis Data Streams. Synthetic events generated by the workload generator are pushed into the Kinesis stream `edtech-events-stream`, which serves as the real-time event bus for the pipeline. Kinesis provides a managed mechanism for continuously ingesting activity data and delivering it to the Lambda function. The stream is active and configured in on-demand mode, which allows the system to absorb changing traffic patterns without manual shard planning, making it appropriate for bursty event workloads.

The screenshot shows the Amazon Kinesis console interface for the 'edtech-events-stream'. The top navigation bar includes the AWS logo, a search bar, and user information. The main content area displays the 'edtech-events-stream' configuration. A 'Data stream summary' section shows the stream is 'Active', has a '1 day' retention period, and is in 'On-demand' capacity mode. Below this, a 'Data stream capacity' section shows the 'Capacity mode' is 'On-demand' and 'Warm throughput' is 'Not set'. Navigation tabs at the bottom include 'Monitoring', 'Configuration' (selected), 'Enhanced fan-out (0)', 'Data viewer', 'Data analytics - new', 'Data stream sharing', and 'EventBridge Pipes'.

Data stream summary			
Status Active	Data retention period 1 day	ARN arn:aws:kinesis:ap-south-1:949925037483:stream/edtech-events-stream	Creation time April 10, 2026 at 12:52 GMT+5:30
Capacity mode On-demand	Maximum record size 1024 KiB		

Data stream capacity	
Capacity mode On-demand	Warm throughput - new Not set

Figure 4: Edtech Events Data Stream

The screenshot shows the 'Data viewer' section of the Amazon Kinesis console for the 'edtech-events-stream'. It displays a list of records for a specific shard. The 'Shard' dropdown is set to 'shardId-000000000000' and the 'Starting position' is 'Trim horizon'. A 'Get records' button is visible. Below, a table shows the first four records, each with a partition key, data (a JSON event), an approximate arrival timestamp, and a sequence number.

Partition key	Data	Approximate arrival timestamp	Sequence number
u069	{"event_id": "0961138f-c6b7-4f2d-a66b-38..."}	April 14, 2026 at 15:10:04 GMT+5:30	49673485979690051085011262774970238...
u097	{"event_id": "857b8f1f-1d8c-4682-8171-18..."}	April 14, 2026 at 15:10:06 GMT+5:30	49673485979690051085011262775654490...
u073	{"event_id": "a0c1be8c-acca-4cf4-b278-06..."}	April 14, 2026 at 15:10:06 GMT+5:30	49673485979690051085011262775866052...
u006	{"event_id": "44932aae-4e9b-466e-9dbd-a..."}	April 14, 2026 at 15:10:07 GMT+5:30	49673485979690051085011262776116300...

Figure 5: Data in Kinesis Stream

Processing Layer

The processing layer is implemented using AWS Lambda, which performs real-time transformation and aggregation of incoming streaming data. As events arrive from Amazon Kinesis Data Streams, the Lambda function processes them in a stateless manner, decoding records, normalizing schema fields, and extracting relevant attributes for downstream analytics.

In addition to basic transformation, the Lambda function maintains operational state using Amazon DynamoDB. It updates active session information and class-level engagement metrics, ensuring that real-time system state is continuously reflected with low latency. Simultaneously, raw event data is persisted into Amazon S3 for long-term storage and historical analysis.

Additionally, the Lambda function publishes operational metrics such as event throughput and processing latency to Amazon CloudWatch. These metrics enable real-time monitoring and provide visibility into system performance.

Overall, the processing layer enables low-latency data transformation and state management without requiring dedicated infrastructure.

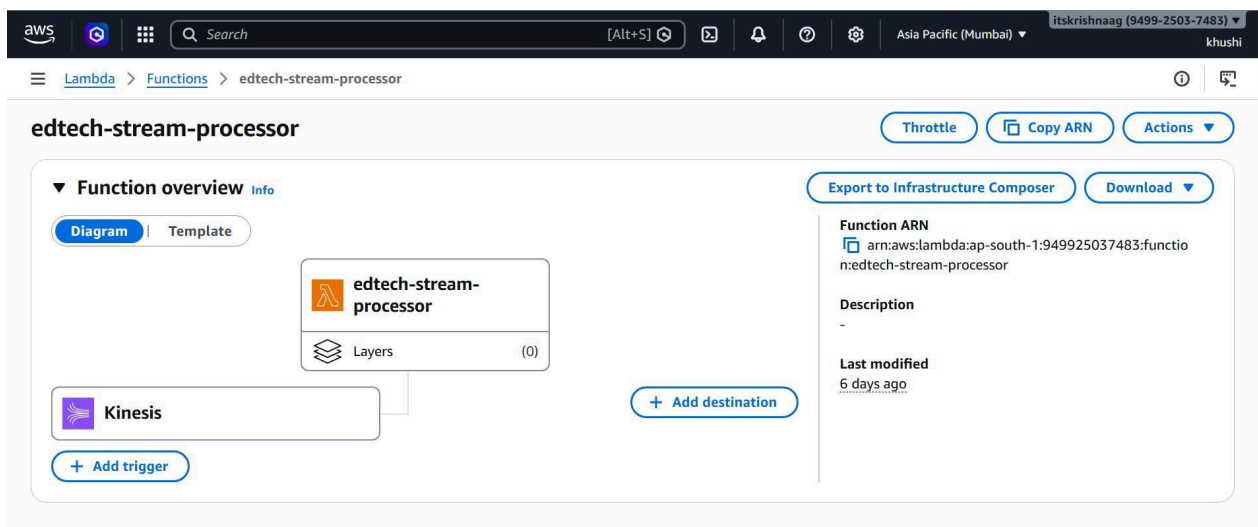


Figure 6: Edtech Stream Processor

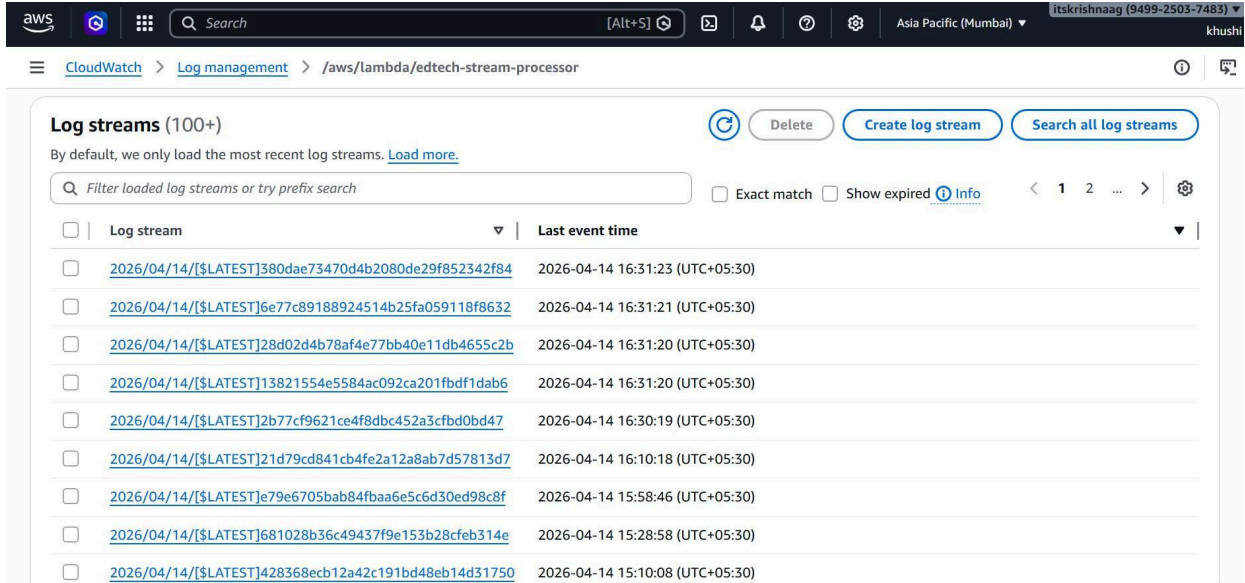


Figure 7: CloudWatch Logs of the Lambda Function

Intelligence Layer

The intelligence component is integrated within the Lambda processing workflow to enable workload-aware behavior. A time-window-based spike detection mechanism is implemented to identify sudden increases in event volume.

Incoming events are aggregated into fixed one-minute buckets, and a rolling baseline is maintained using an Exponential Moving Average (EMA). This approach captures recent workload trends while smoothing short-term fluctuations.

Spike detection is performed by comparing the current minute's event count against a dynamically computed threshold derived from the EMA baseline. A spike is identified when the current workload exceeds a configurable multiple of the rolling average.

To ensure stability and prevent bias, the EMA is updated only once per time window, avoiding multiple partial updates within the same interval. This improves the robustness of spike detection and reduces sensitivity to noise.

The detected spike indicators are also published as metrics, allowing the system to monitor workload variations in real time.

Storage and Metadata Layer

The storage layer consists of Amazon S3 and DynamoDB, serving complementary purposes. Amazon S3 is used to store raw event data in its original JSON format, ensuring durability and enabling future reprocessing or historical analysis. DynamoDB is used to store processed and aggregated data, such as active sessions and class-level metrics, providing low-latency access to real-time information.

To enable structured querying over raw data stored in S3, AWS Glue is used as a metadata catalog service. A Glue crawler automatically scans the S3 bucket, infers the schema of the JSON data, and creates corresponding tables in the Glue Data Catalog. These tables are then used by Athena to query raw event data efficiently without requiring manual schema definition.

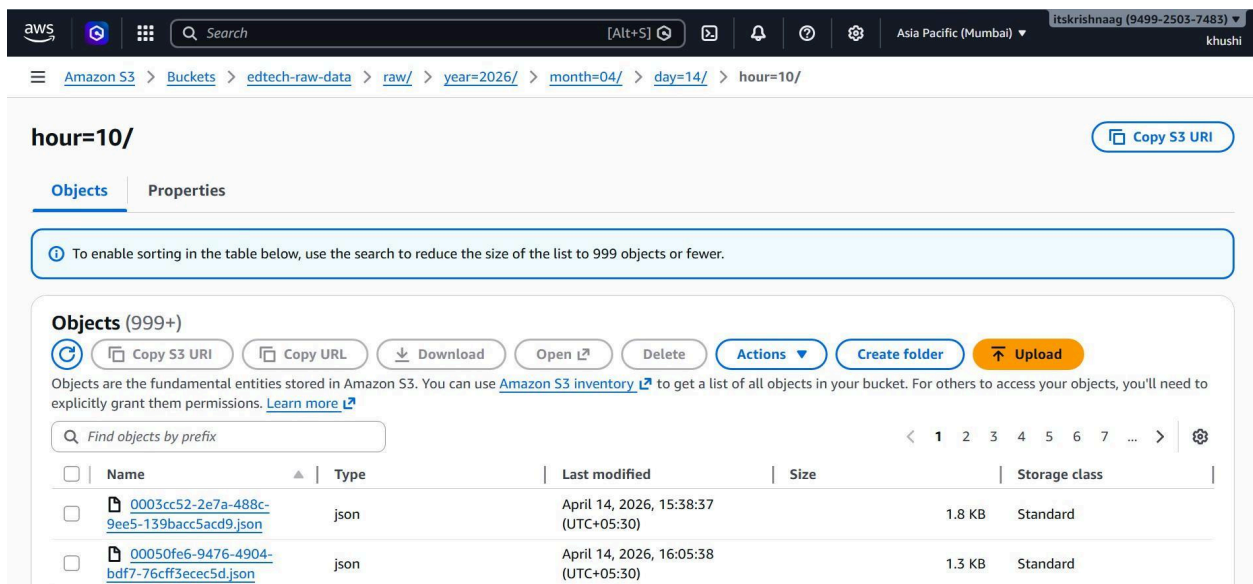


Figure 8: Raw Data Stored in S3

Table: class_metrics - Items returned (10)

Scan started on April 14, 2026, 16:16:03

	class_id (...)	active_user_count	course_name	last_updated	region_breakdown
☐	csg244	19236	Web Develop...	2026-04-14T1...	{ "Bengaluru": { "N": "2748" }, "Chennai": { "N": "5342" }, "Delhi": { "N": "113" } }
☐	csg415	13582	Linear Algebra	2026-04-14T1...	{ "Bengaluru": { "N": "216" }, "Chennai": { "N": "4009" }, "Delhi": { "N": "113" } }
☐	csg527	15065	Advanced Alg...	2026-04-14T1...	{ "Bengaluru": { "N": "1495" }, "Chennai": { "N": "2748" }, "Delhi": { "N": "113" } }
☐	csg302	28105	Python for Be...	2026-04-14T1...	{ "Bengaluru": { "N": "1416" }, "Chennai": { "N": "4533" }, "Delhi": { "N": "113" } }
☐	csg499	11359	Statistics & Pr...	2026-04-14T1...	{ "Bengaluru": { "N": "4091" }, "Chennai": { "N": "113" }, "Delhi": { "N": "113" } }
☐	csg175	21716	Cloud Compu...	2026-04-14T1...	{ "Bengaluru": { "N": "2763" }, "Chennai": { "N": "2727" }, "Delhi": { "N": "113" } }
☐	csg388	15683	Database Syst...	2026-04-14T1...	{ "Bengaluru": { "N": "2843" }, "Chennai": { "N": "1471" }, "Delhi": { "N": "113" } }
☐	csg511	5211	Machine Lear...	2026-04-14T1...	{ "Bengaluru": { "N": "103" }, "Chennai": { "N": "1206" }, "Delhi": { "N": "113" } }
☐	csg610	235	Deep Learning	2026-04-14T1...	{ "Bengaluru": { "N": "97" }, "Chennai": { "N": "118" }, "Delhi": { "N": "113" } }

Figure 9: Glue Crawler for scanning S3 Bucket

Table details

Name raw_raw	Classification JSON	Deprecated -
Database edtech-analytics	Location s3://edtech-raw-data/raw/	Column statistics No statistics
Description -	Connection -	
Last updated April 14, 2026 at 10:51:47		

► **Advanced properties**

Schema (34)

[Edit schema as JSON](#) [Edit schema](#)

Figure 10: Raw Data Table in Glue Data Catalog

Table: active_sessions - Items returned (50)
Scan started on April 14, 2026, 16:12:09

session_id (String)	class_id	device	last_seen	region	start_time	status	ttl (TTL)
s61595	csg388		2026-04-0...			in_class	
s15684			2026-04-0...			in_session	
s68895	csg527		2026-04-1...			in_class	
s46500	csg527		2026-04-0...			in_class	
s45758	csg302		2026-04-0...			in_class	
s52025	csg175		2026-04-0...			in_class	
s17208	csg175		2026-04-0...			in_class	
s15969	csg302		2026-04-0...			in_class	
s24708	csg221	desktop	2026-04-1...	Bengaluru	2026-04-14...	in_session	177616649

Figure 11: DynamoDB Active Sessions Table

Table: class_metrics - Items returned (10)
Scan started on April 14, 2026, 16:16:03

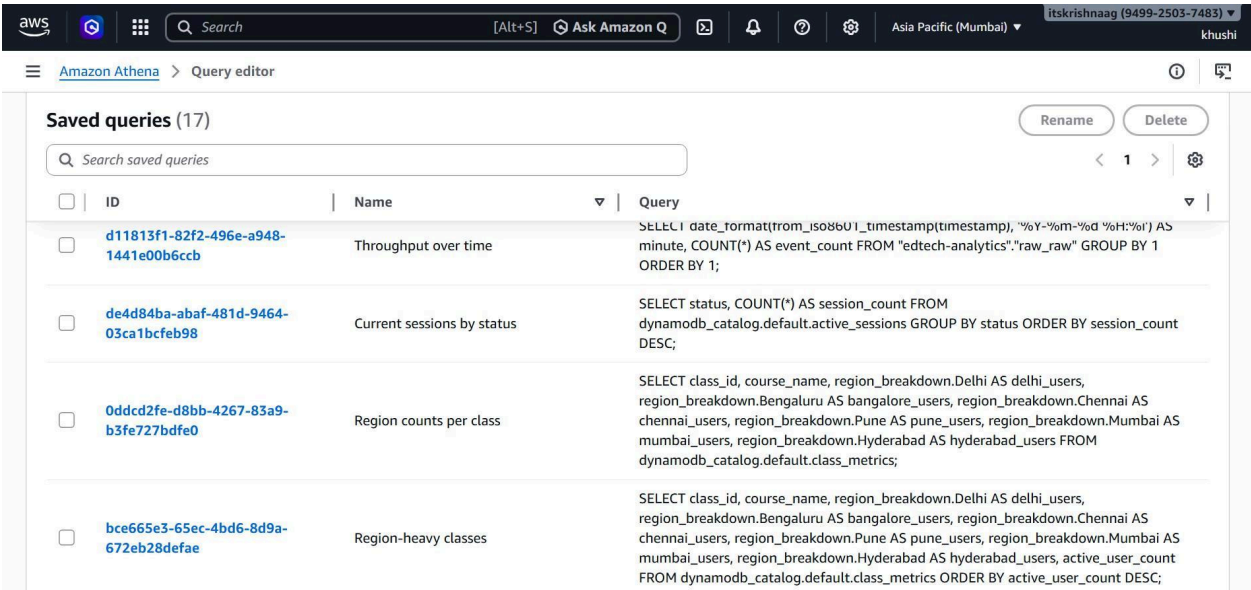
class_id	active_user_count	course_name	last_updated	region_breakdown
csg244	19236	Web Develop...	2026-04-14T1...	{ "Bengaluru": { "N": "2748" }, "Chennai": { "N": "5342" }, "Del...
csg415	13582	Linear Algebra	2026-04-14T1...	{ "Bengaluru": { "N": "216" }, "Chennai": { "N": "4009" }, "Delhi...
csg527	15065	Advanced Alg...	2026-04-14T1...	{ "Bengaluru": { "N": "1495" }, "Chennai": { "N": "2748" }, "Del...
csg302	28105	Python for Be...	2026-04-14T1...	{ "Bengaluru": { "N": "1416" }, "Chennai": { "N": "4533" }, "Del...
csg499	11359	Statistics & Pr...	2026-04-14T1...	{ "Bengaluru": { "N": "4091" }, "Chennai": { "N": "113" }, "Delhi...
csg175	21716	Cloud Compu...	2026-04-14T1...	{ "Bengaluru": { "N": "2763" }, "Chennai": { "N": "2727" }, "Del...
csg388	15683	Database Syst...	2026-04-14T1...	{ "Bengaluru": { "N": "2843" }, "Chennai": { "N": "1471" }, "Del...
csg511	5211	Machine Lear...	2026-04-14T1...	{ "Bengaluru": { "N": "103" }, "Chennai": { "N": "1206" }, "Delhi...
csg610	235	Deep Learning	2026-04-14T1...	{ "Bengaluru": { "N": "97" }, "Chennai": { "N": "118" }, "Delhi": ...

Figure 12: DynamoDB Class Metrics Table

Query Layer

The query layer is implemented using Amazon Athena with federated query support. Athena enables SQL-based querying over both S3 and DynamoDB data sources. Through federated

queries, DynamoDB tables are made accessible in Athena via a connector, allowing unified querying without data duplication. This facilitates efficient analysis by combining real-time aggregated data with raw event data when required.



The screenshot shows the Amazon Athena Query Editor interface. At the top, there's a navigation bar with the AWS logo, a search bar, and user information. Below this, the 'Saved queries (17)' section is visible. It contains a table with columns for ID, Name, and Query. Four queries are listed:

ID	Name	Query
d11813f1-82f2-496e-a948-1441e00b6ccb	Throughput over time	SELECT date_format(from_iso8601_timestamp(timestamp), '%Y-%m-%d %H:%i') AS minute, COUNT(*) AS event_count FROM "edtech-analytics"."raw_raw" GROUP BY 1 ORDER BY 1;
de4d84ba-abaf-481d-9464-03ca1bcfeb98	Current sessions by status	SELECT status, COUNT(*) AS session_count FROM dynamodb_catalog.default.active_sessions GROUP BY status ORDER BY session_count DESC;
0ddcd2fe-d8bb-4267-83a9-b3fe727bdf0	Region counts per class	SELECT class_id, course_name, region_breakdown.Delhi AS delhi_users, region_breakdown.Bengaluru AS bangalore_users, region_breakdown.Chennai AS chennai_users, region_breakdown.Pune AS pune_users, region_breakdown.Mumbai AS mumbai_users, region_breakdown.Hyderabad AS hyderabad_users FROM dynamodb_catalog.default.class_metrics;
bce665e3-65ec-4bd6-8d9a-672eb28defae	Region-heavy classes	SELECT class_id, course_name, region_breakdown.Delhi AS delhi_users, region_breakdown.Bengaluru AS bangalore_users, region_breakdown.Chennai AS chennai_users, region_breakdown.Pune AS pune_users, region_breakdown.Mumbai AS mumbai_users, region_breakdown.Hyderabad AS hyderabad_users, active_user_count FROM dynamodb_catalog.default.class_metrics ORDER BY active_user_count DESC;

Figure 13: Athena queries on the S3 and DynamoDB data

Visualization Layer

The visualization layer is built using Amazon QuickSight, which provides interactive dashboards for analyzing system metrics. QuickSight connects to Athena as a data source, enabling the creation of visualizations such as user activity trends, event throughput, regional distribution, and class engagement metrics. The dashboards offer an intuitive interface for exploring insights and monitoring system behavior in near real-time.

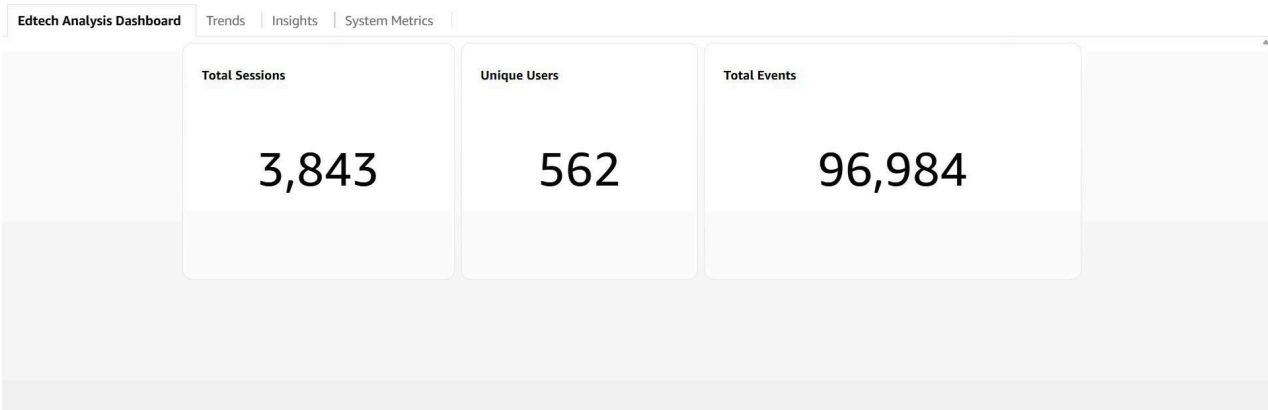


Figure 14: Edtech Analysis Dashboard

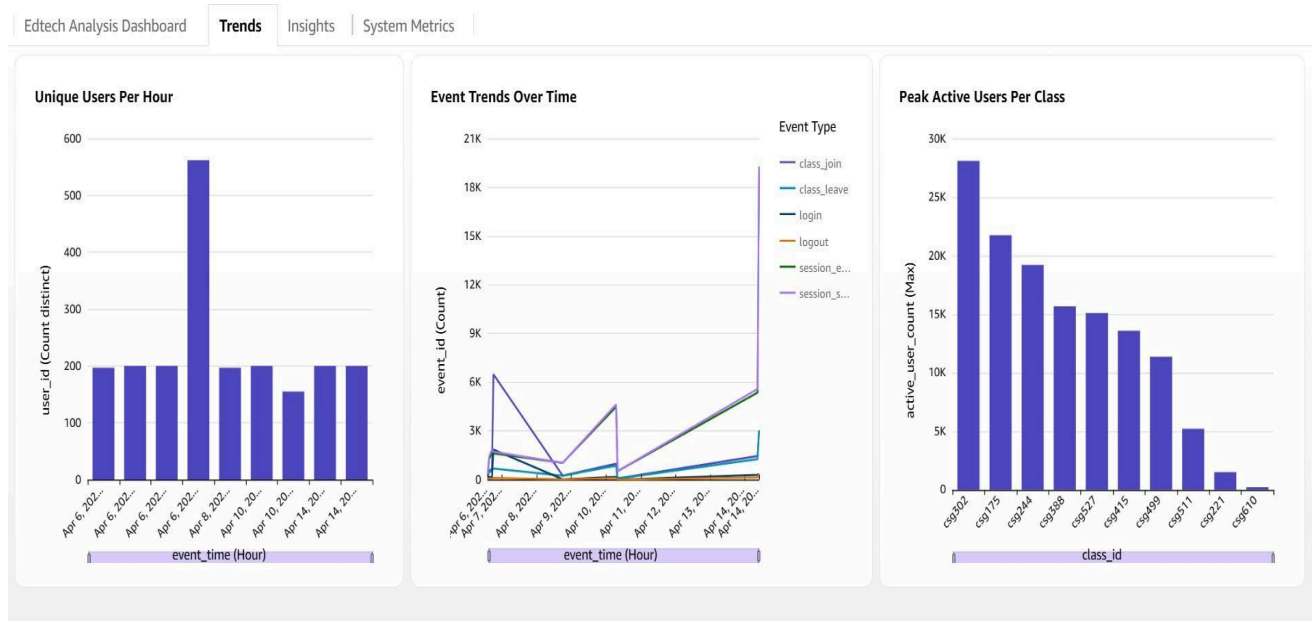


Figure 15: Trends Dashboard

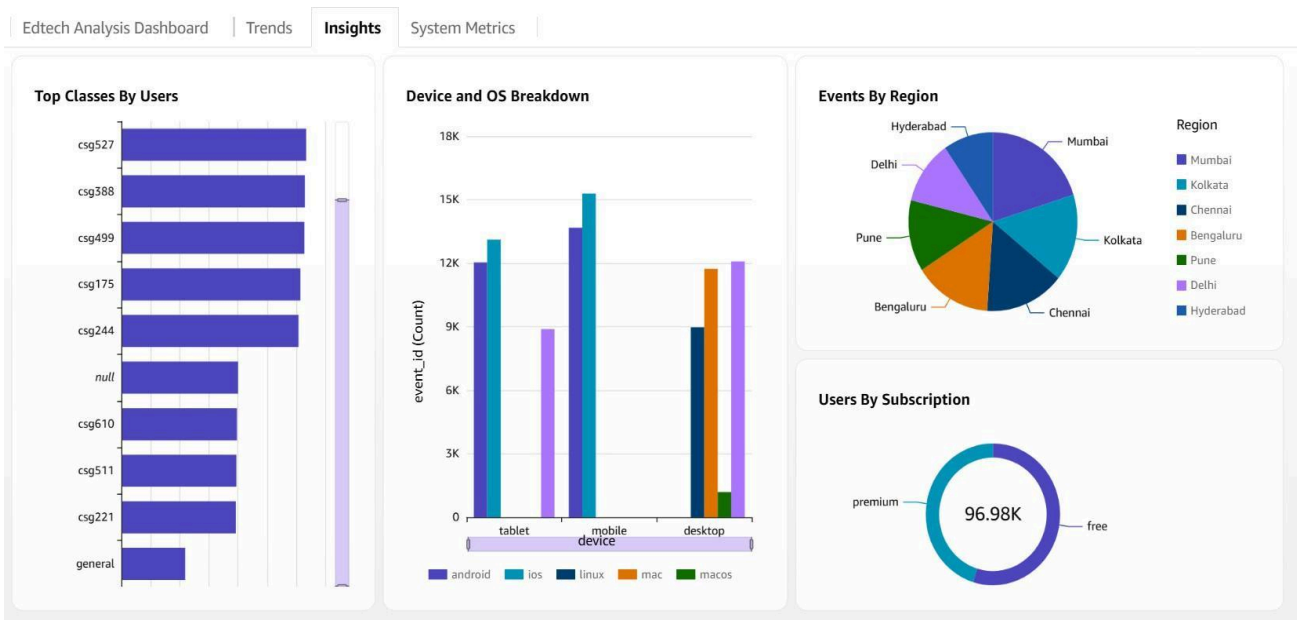


Figure 16: Insights Dashboard

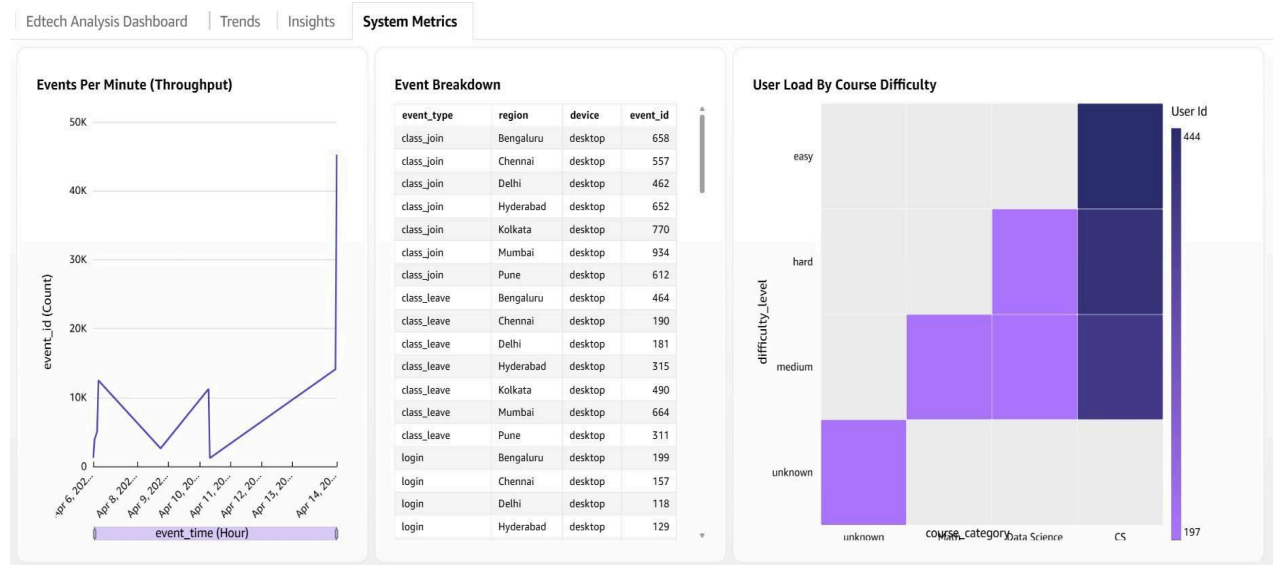


Figure 17: System Metrics Dashboard

Challenges Faced in the Serverless Architecture

While implementing the serverless analytics pipeline, several challenges were encountered.

One of the main challenges was the **stateless nature of AWS Lambda**. Since each invocation runs independently, it was difficult to maintain continuous session information and real-time metrics. This required the use of DynamoDB to store and update state, which added complexity to the design.

Another challenge was **debugging and monitoring**. Since the system is distributed across multiple services, tracing the flow of data required checking CloudWatch logs for each Lambda invocation. This made debugging more time-consuming compared to a centralized system.

Handling **high event rates** was also difficult. As the number of incoming events increased, Lambda invocations increased proportionally, leading to higher costs and potential performance issues. Understanding and managing this behavior required careful testing.

Implementing **spike detection** in a stateless environment was another challenge. The system needed to group events into time windows and maintain a rolling baseline using DynamoDB, which required careful handling to ensure correct updates.

Finally, integrating multiple AWS services such as Kinesis, Lambda, S3, DynamoDB, and Athena required proper configuration and permissions, which made the setup process more complex.

These challenges highlighted the limitations of the serverless approach and motivated the design of the second pipeline.

Limitation of Serverless Architecture

The serverless baseline demonstrated the viability of a cloud-native analytics workflow; however, several limitations were observed under high-throughput continuous workloads. One major concern is the **cost model of AWS Lambda**, where pricing is based on per- invocation and execution duration. As the event rate increases, the number of invocations grows linearly, leading to significantly higher operational costs.

Additionally, the **stateless nature of Lambda functions** restricts the implementation of advanced streaming operations such as windowed aggregations and stateful processing without relying on external coordination mechanisms. This introduces additional complexity and overhead. Furthermore, the use of **Kinesis Data Streams imposes shard-level throughput limits**, which restrict scalability and make it difficult to handle sudden bursts in data volume efficiently.

Motivation for Alternative Architecture

To overcome these limitations, an alternative architecture was designed with a focus on cost efficiency, scalability, and processing flexibility. The key objective was to move away from per-invocation billing and adopt a model that supports **persistent, long-running computation**, making it more suitable for continuous streaming workloads.

This led to the adoption of a server-based streaming stack deployed on EC2 instances, while still retaining AWS-managed services such as S3, DynamoDB, and IAM for storage and security. By leveraging open-source technologies like Apache Kafka for ingestion and PySpark for processing, the system achieves **greater control over resource allocation, state management, and processing logic**, enabling more efficient handling of high- throughput data streams and improved system performance.

Pipeline 2: Distributed Streaming Architecture (Optimized)

The second pipeline is designed as a distributed, infrastructure-driven real-time analytics system that improves upon the limitations of the serverless approach. It is built to handle high-throughput, bursty workloads while maintaining low latency, cost efficiency, and scalability.

The architecture integrates Apache Kafka for ingestion, PySpark Structured Streaming for real-time processing, Redis for low-latency analytics, Amazon S3 for cold storage, DynamoDB for structured aggregation, and Prometheus with Grafana for monitoring. In addition, Kubernetes (k3s) with KEDA is used to enable event-driven horizontal scaling of storage operations. All infrastructure is provisioned using Terraform, ensuring reproducibility and consistency.

This pipeline follows a Lambda-style architecture, combining both streaming and batch processing paths to support real-time insights and historical analytics simultaneously.

Architecture Overview

The system is deployed across two EC2 instances in the ap-south-1 region. The first instance hosts Apache Kafka, which acts as the central event ingestion layer. It maintains two topics: edtech-events for the primary data stream and edtech-priority for surge handling.

The second instance hosts the processing and monitoring components. PySpark Structured Streaming consumes events from Kafka and performs real-time analytics. Redis is used to store live metrics for dashboard visualization.

A Kubernetes cluster (k3s) is deployed on the same instance to run scalable storage worker pods. These pods consume data from Kafka and perform high-throughput writes to Amazon S3 and DynamoDB. KEDA enables automatic scaling of these pods based on Kafka consumer lag.

Prometheus collects system and application metrics, while Grafana provides dashboards for visualization. Terraform is used to provision all infrastructure components, including EC2 instances, IAM roles, networking, and monitoring setup, ensuring a fully automated deployment.

Parameter	Value
Normal event rate	30 events/second
Peak event rate	300 events/second
Kafka partitions (main topic)	10
Kafka partitions (priority topic)	3
Spark micro-batch interval	10 seconds
KEDA scaling range	1 to 10 pods
Storage worker memory	250 MB per pod
Batch job execution time	118 seconds
Failure recovery time	5 seconds

Table 1: System Configuration and Key Parameters

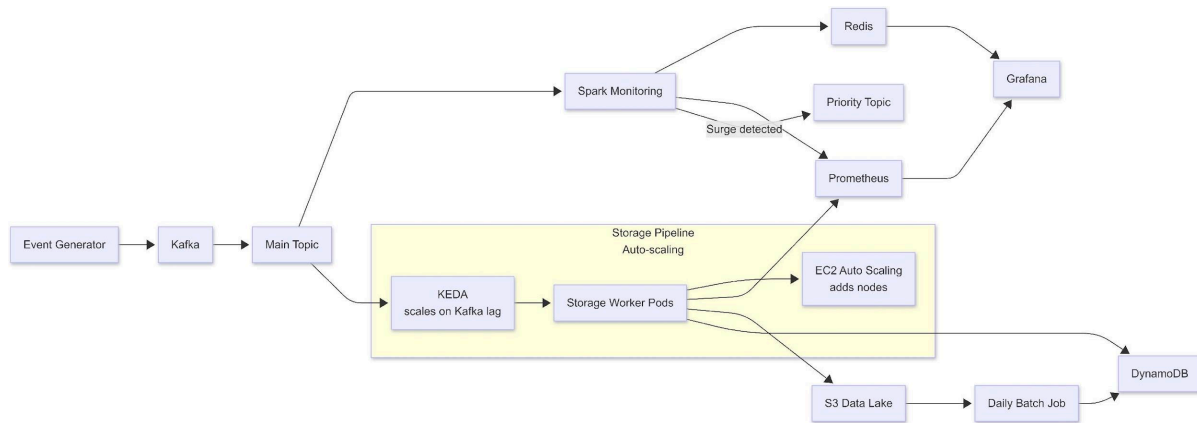


Figure 18: Pipeline 2 Architecture

Data Generation Layer

The pipeline begins with a custom event generator that simulates user interactions on an EdTech platform. The generator produces structured JSON events representing actions such as session start, course participation, and content interaction.

The generator operates at configurable event rates and supports multiple workload modes, including steady and burst traffic patterns. It produces approximately 30 events per second under normal conditions, with spikes up to 3× or higher during stress testing.

These events are directly published to Kafka, enabling controlled experimentation and realistic simulation of production workloads.

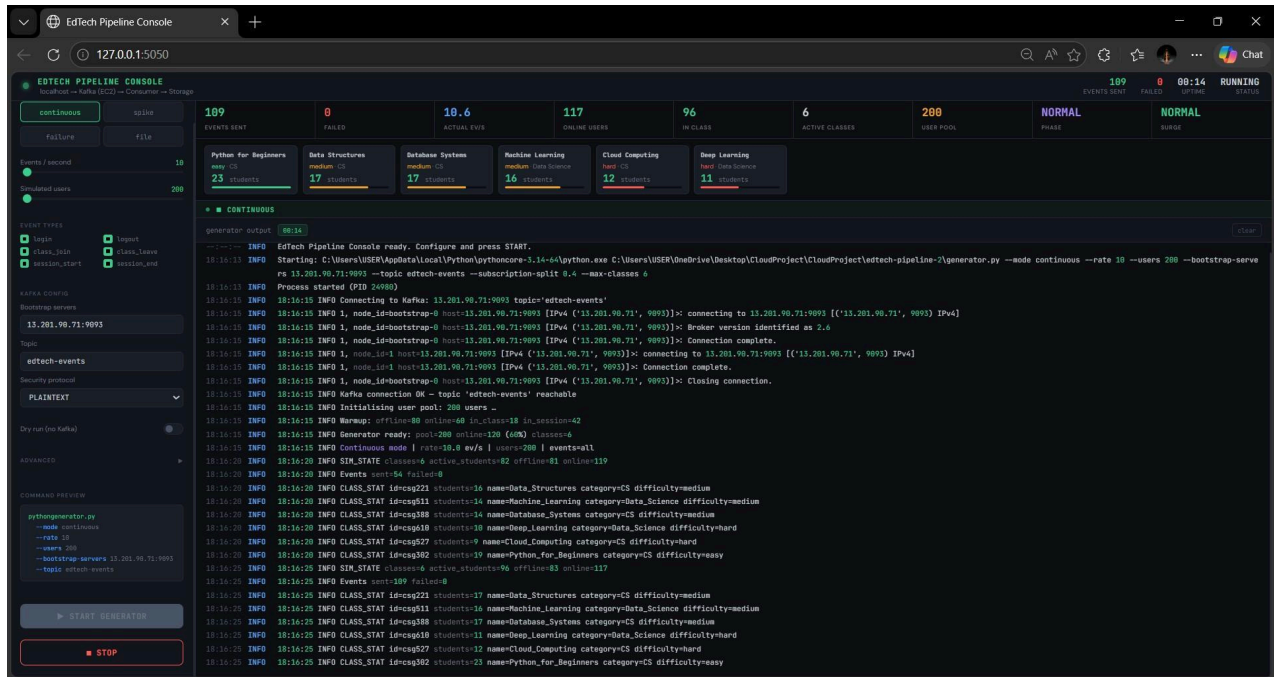


Figure 19: Generator Console

Data Ingestion Layer

The ingestion layer is implemented using Apache Kafka, which provides a durable and scalable event streaming platform. Events are written to the edtech-events topic, which is partitioned into 10 partitions to enable parallel processing.

Kafka ensures fault tolerance by persisting events and allowing multiple consumer groups to read the same data independently. During surge conditions, overflow events are redirected to a secondary topic, edtech-priority, enabling load isolation and improved system stability.

This design allows the pipeline to handle bursty workloads without data loss while maintaining consistent ingestion performance.

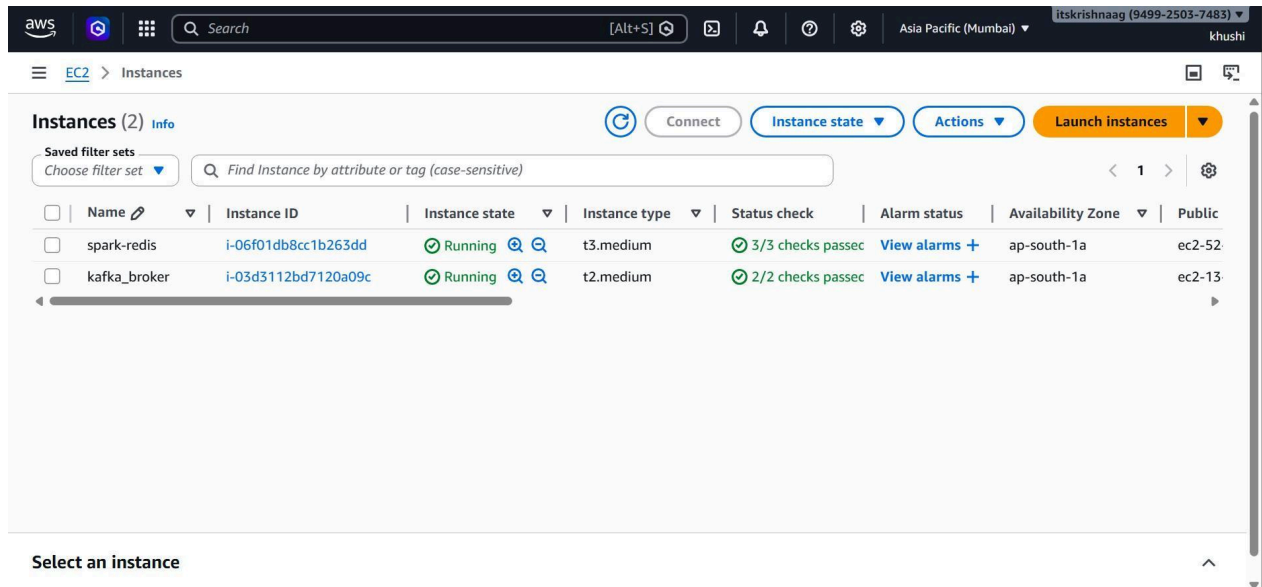


Figure 20: Running EC2 Instances of Kafka and Spark/Redis

Processing Layer

The processing layer is implemented using PySpark Structured Streaming, which consumes events from Kafka in 10-second micro-batches. It performs real-time transformations and computes metrics such as active users, engagement levels, and dropout rates.

The results of these computations are stored in Redis, which serves as a hot storage layer for real-time dashboards. By focusing only on computation and analytics, the processing layer ensures low latency and efficient handling of streaming data.

Intelligence Layer

The intelligence layer is embedded within the processing workflow and introduces adaptive behavior into the system. It consists of four key components across two tiers: Adaptive-BatchOptimizer, SurgeDetector, KEDA-based scaling, and EC2 Auto Scaling.

The AdaptiveBatchOptimizer buffers events and writes them to Amazon S3 only when a threshold is reached or a time limit is exceeded. This significantly reduces storage costs while maintaining acceptable latency for batch analytics.

The SurgeDetector monitors event rates using a rolling baseline and detects spikes when

the current rate exceeds 1.5 times the average. During such conditions, overflow events are routed to the edtech-priority topic, preventing system overload and maintaining SLA compliance.

At the orchestration level, KEDA dynamically scales storage worker pods based on Kafka consumer lag, while EC2 Auto Scaling provisions additional compute resources when system utilization increases.

These mechanisms collectively enable cost-efficient, scalable, and SLA-aware data processing.

Component	Layer	Function
AdaptiveBatchOptimizer	Processing Intelligence	Reduces S3 write operations by buffering events and flushing based on thresholds
SurgeDetector	Processing Intelligence	Detects workload spikes and routes overflow traffic to priority topic
KEDA Scaling	Orchestration Intelligence	Dynamically scales storage worker pods based on Kafka lag
EC2 Auto Scaling	Infrastructure Intelligence	Provisions additional compute resources under high load

Table 2: Intelligence Layer Components and Responsibilities

Parameter	Spark Value	k8s Pod Value	Description
FLUSH_EVENTS	10,000	5,000	Number of events required to trigger S3 write
FLUSH_SECS	300	300	Maximum buffering time before forced flush

Table 3: AdaptiveBatchOptimizer Configuration Parameters

Scalable Storage Layer

The storage layer is implemented using Kubernetes-based storage worker pods. Each pod runs a lightweight Python consumer that reads events from Kafka and performs storage operations.

Events are buffered and written to Amazon S3 in partitioned Parquet format, while aggregated metrics are stored in DynamoDB tables such as UserStats and CourseStats. Each worker requires approximately 250 MB of memory, enabling efficient horizontal scaling.

KEDA monitors Kafka consumer lag and dynamically adjusts the number of pods between 1 and 10, ensuring that storage throughput scales with workload demand.

This separation of storage from processing allows the system to achieve high throughput and improved resource utilization.

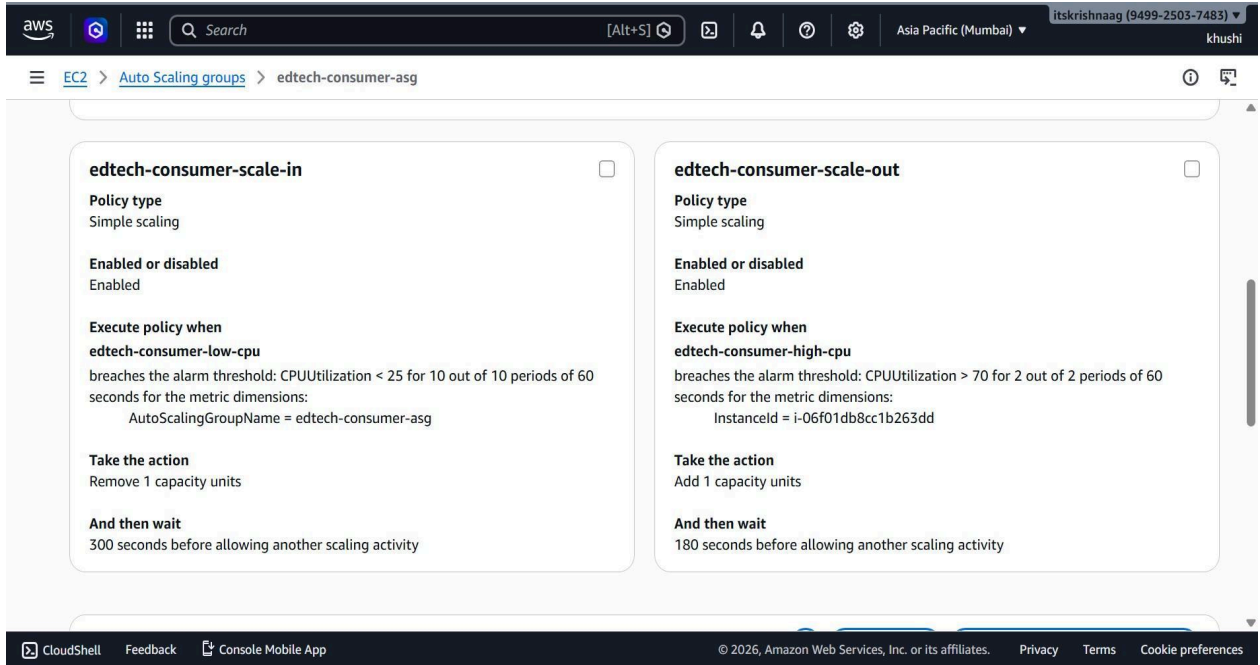


Figure 21: Dynamic Scaling Policies

Scaling Method	Trigger Metric	Behavior
KEDA	Kafka Consumer Lag	Scales pods based on back-log (1–10 pods)
EC2 Auto Scaling	CPU Utilization	Adds/removes instances when CPU exceeds threshold

Table 4: Two-Tier Auto-Scaling Mechanism

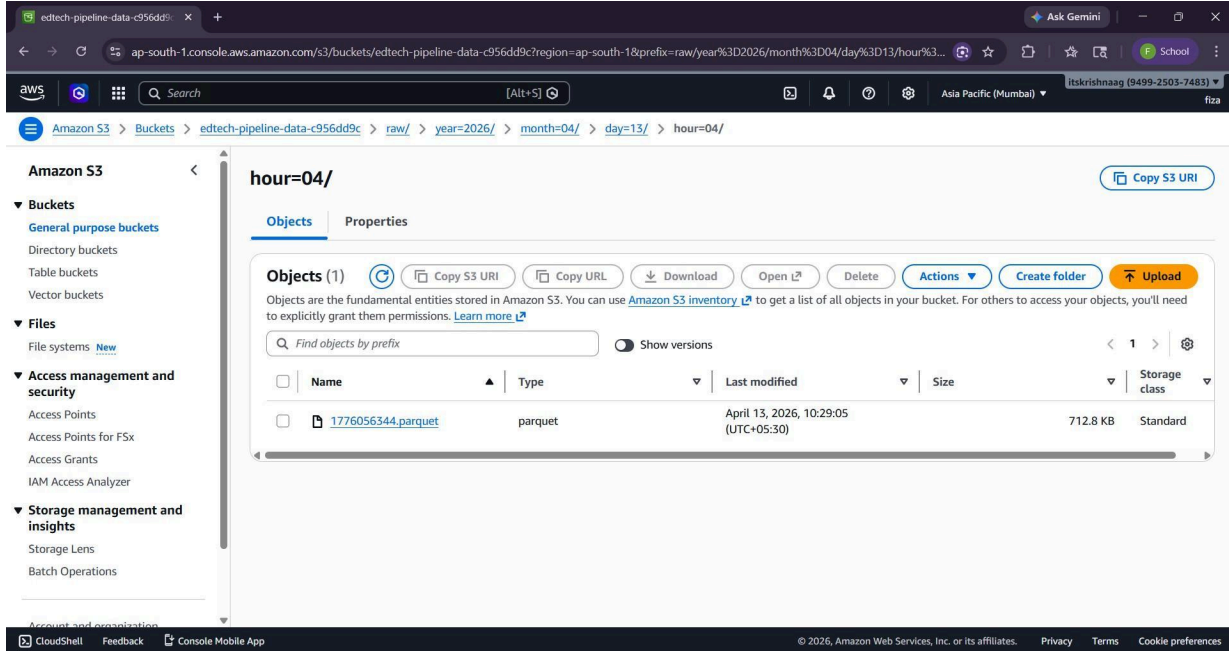


Figure 22: Processed Data in S3 in partitioned Parquet format

Batch Processing Layer

A batch processing job runs periodically to generate daily analytics. It reads Parquet data from Amazon S3 and computes aggregated metrics such as daily active users, pass rates, and engagement statistics.

The results are stored in the DailyReports table in DynamoDB, enabling efficient querying and reporting. The batch job processes over 267,000 events in approximately 118 seconds, demonstrating efficient large-scale data processing.

Workflow orchestration : The project specification Sec. 4.3 lists Airflow or Dagster as candidate orchestration tools. For this deployment, the batch job is scheduled using a Linux cron entry (0 0 * * * /home/ubuntu/run_batch_reports.sh) rather than a full Airflow or Dagster deployment. The choice was driven by resource constraints — EC2-2 is a single t3.medium (4 GB RAM) hosting Spark, Redis, Prometheus, Grafana, and the k3s control plane. An Airflow scheduler plus worker processes would consume approximately 800 MB–1.5 GB of RAM, which is not available without scaling the node to t3.large. A Dagster-compatible DAG file (dags/batch_reports_dag.py) was authored for portability: if this pipeline were redeployed on a multi-node cluster or migrated to a managed Airflow service (MWAA), the existing DAG would integrate without modification. For a single-node deployment, cron provides equivalent scheduling guarantees at zero operational cost.

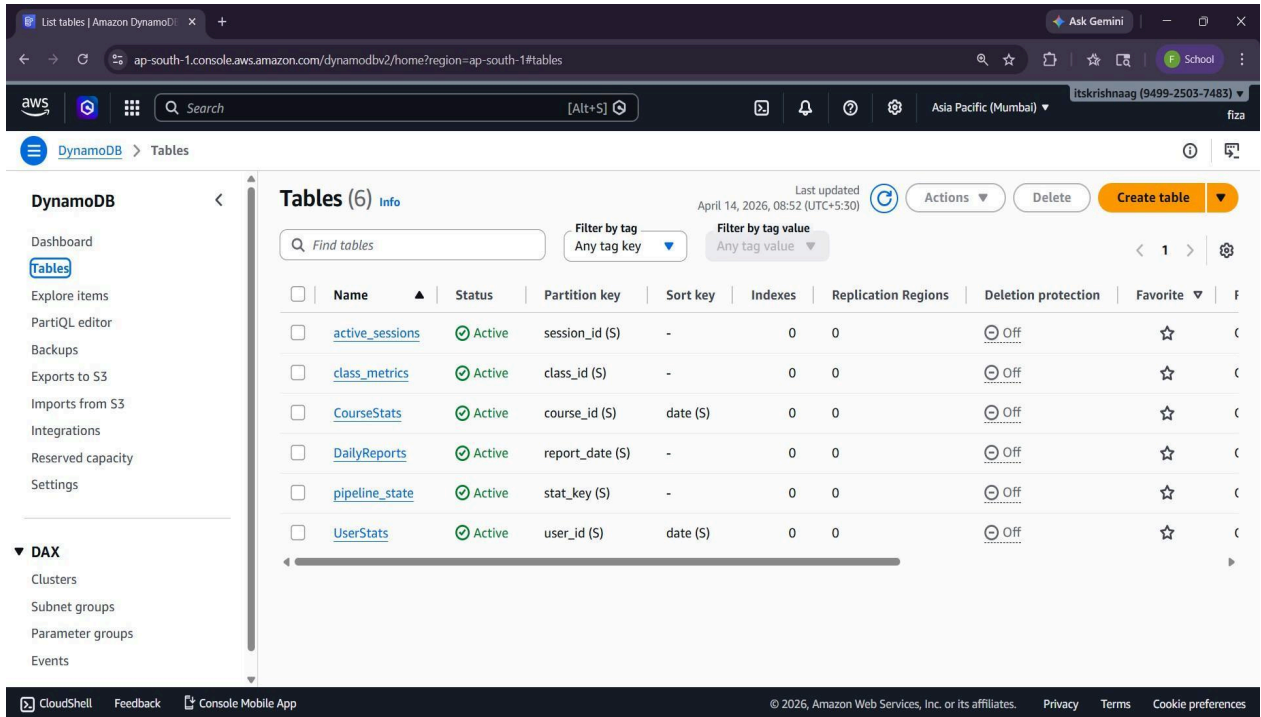


Figure 23: UserStats, CourseStats and DailyReports Tables in DynamoDB

A separate batch job processes S3 data daily and writes summarized reports to DynamoDB.

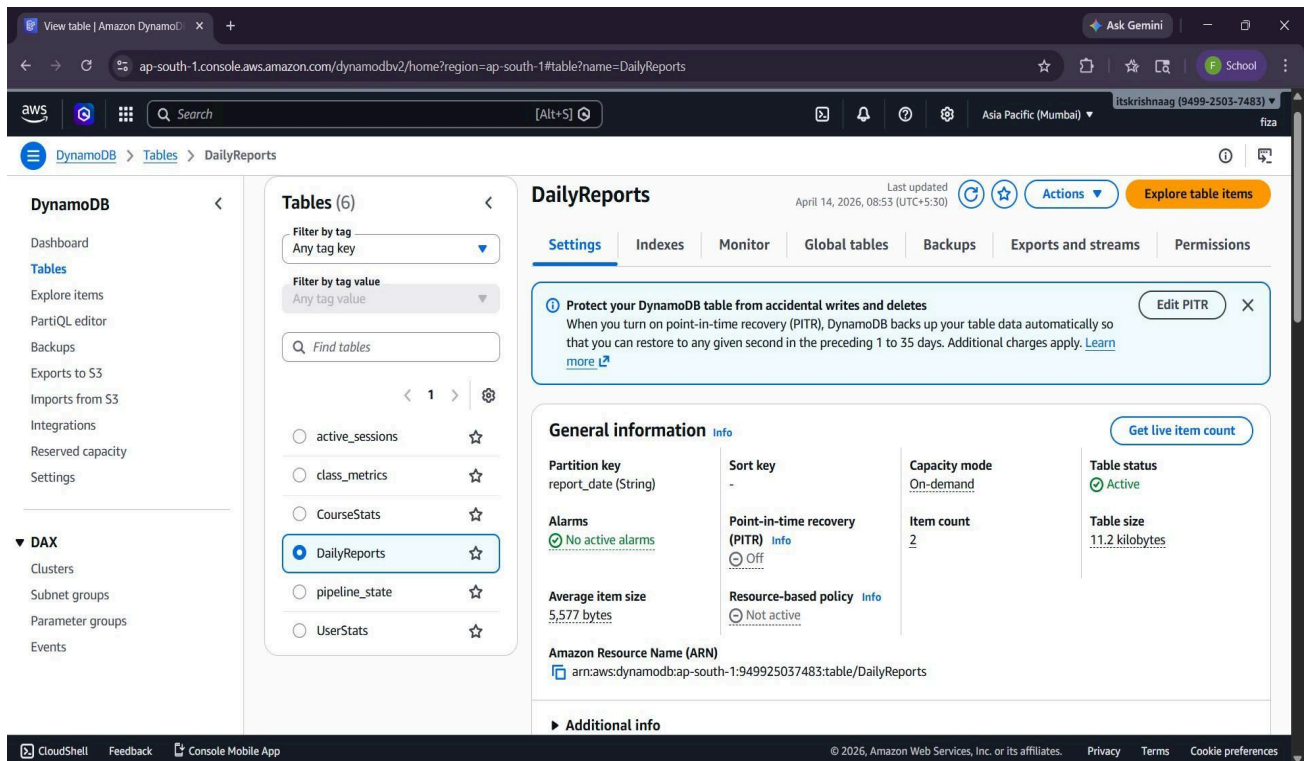


Figure 24: Daily Reports in DynamoDB

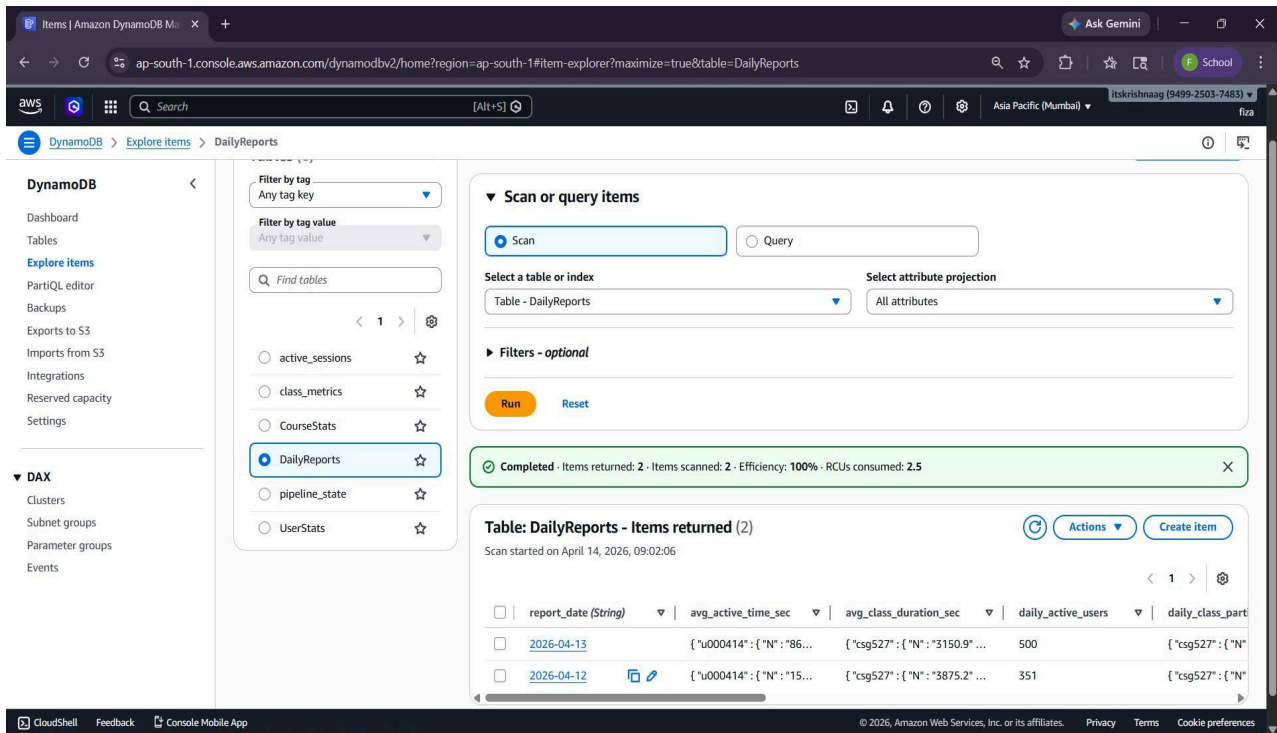


Figure 25: Items of Daily Reports

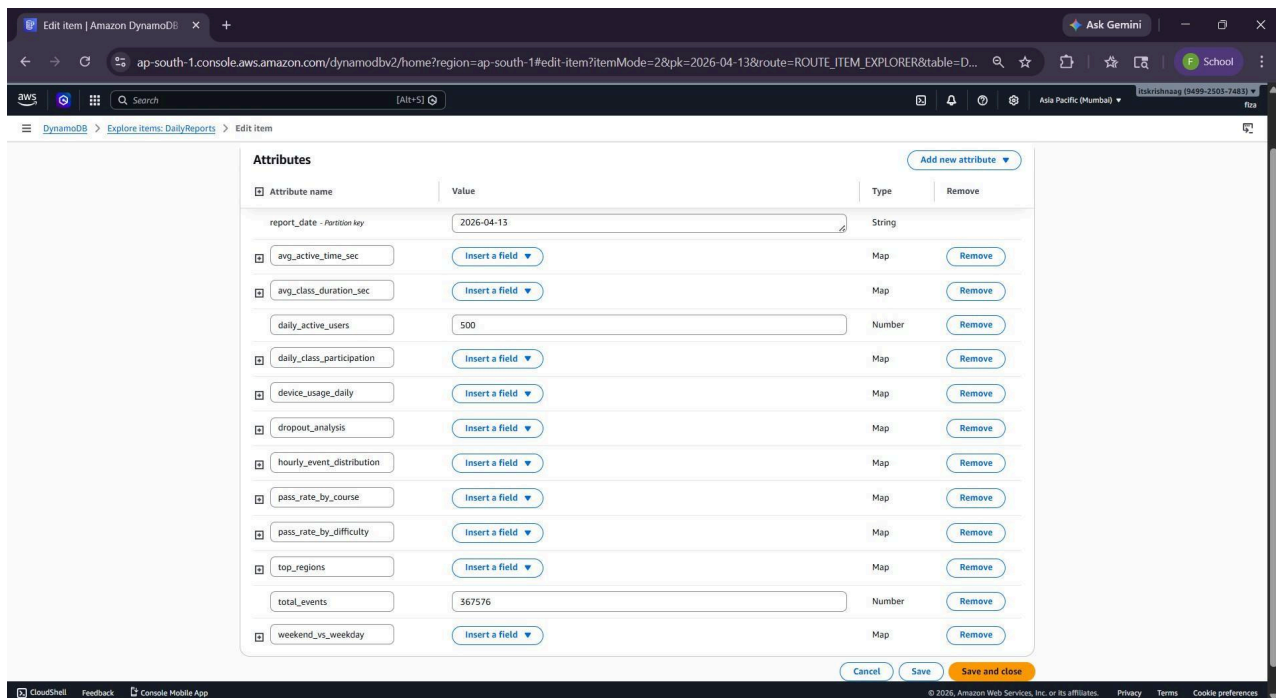


Figure 26: Attributes of Daily Reports Items

Monitoring and Metrics Layer

The pipeline includes a comprehensive monitoring layer implemented using Prometheus. The Spark application exposes a metrics endpoint that provides both system-level and application-level metrics, including:

- Events processed
- Kafka consumer lag
- Batch processing duration
- Active users and engagement metrics
- Surge detection status
- S3 cost reduction statistics

In addition to the conceptual metrics, the system exposes real-time values through a Prometheus-compatible endpoint. A sample snapshot of the metrics includes:

- pipeline events processed total: Total number of events processed from Kafka (e.g., 87621 events)
- pipeline kafka consumer lag: Number of unprocessed events (e.g., 1760), indicating system backpressure
- pipeline active users gauge: Number of active users in the current batch (e.g., 353 users)
- pipeline dropout rate gauge: Percentage of users leaving sessions (e.g., 2.7%)
- pipeline avg engagement gauge: Average engagement score (e.g., 0.693)
- pipeline surge detected: Indicates whether a traffic surge is active (0 = normal, 1 = surge)

These metrics provide both system-level insights (such as processing lag and throughput) and business-level insights (such as user engagement and dropout behavior).

```

# HELP pipeline_batch_duration_seconds_created Seconds spent processing each micro-batch
# TYPE pipeline_batch_duration_seconds_created gauge
pipeline_batch_duration_seconds_created 1.7761558479227023e+09
# HELP pipeline_events_processed_total Cumulative events processed from Kafka
# TYPE pipeline_events_processed_total counter
pipeline_events_processed_total 87621.0
# HELP pipeline_events_processed_created Cumulative events processed from Kafka
# TYPE pipeline_events_processed_created gauge
pipeline_events_processed_created 1.7761558479227576e+09
# HELP pipeline_kafka_consumer_lag Estimated number of events behind latest Kafka offset
# TYPE pipeline_kafka_consumer_lag gauge
pipeline_kafka_consumer_lag 1760.0
# HELP pipeline_active_users_gauge Distinct logins seen in the last micro-batch
# TYPE pipeline_active_users_gauge gauge
pipeline_active_users_gauge 353.0
# HELP pipeline_dropout_rate_gauge Dropout rate % (voluntary class_leave / class_join) in last batch
# TYPE pipeline_dropout_rate_gauge gauge
pipeline_dropout_rate_gauge 2.7
# HELP pipeline_avg_engagement_gauge Average engagement score from session_end events in last batch
# TYPE pipeline_avg_engagement_gauge gauge
pipeline_avg_engagement_gauge 0.693
# HELP pipeline_surge_detected 1 if a traffic surge is currently active, 0 otherwise
# TYPE pipeline_surge_detected gauge
pipeline_surge_detected 0.0
# HELP pipeline_course_joins_class_join count per course in last batch window
# TYPE pipeline_course_joins gauge
pipeline_course_joins{course="csg221"} 15.0
pipeline_course_joins{course="csg415"} 29.0
pipeline_course_joins{course="csg388"} 23.0
pipeline_course_joins{course="csg511"} 33.0
pipeline_course_joins{course="csg302"} 40.0
pipeline_course_joins{course="csg499"} 13.0
pipeline_course_joins{course="csg244"} 18.0
pipeline_course_joins{course="csg610"} 20.0
pipeline_course_joins{course="csg527"} 18.0
pipeline_course_joins{course="csg175"} 21.0
# HELP pipeline_device_events Event count by device type in last batch window
# TYPE pipeline_device_events gauge
pipeline_device_events{device="tablet"} 183.0
pipeline_device_events{device="mobile"} 486.0
pipeline_device_events{device="desktop"} 249.0
# HELP pipeline_region_users Online users by region in last batch window
# TYPE pipeline_region_users gauge
pipeline_region_users{region="Hyderabad"} 2.0
pipeline_region_users{region="Bengaluru"} 2.0
pipeline_region_users{region="Kolkata"} 1.0
pipeline_region_users{region="Chennai"} 2.0
pipeline_region_users{region="Pune"} 1.0
pipeline_region_users{region="Mumbai"} 1.0
pipeline_region_users{region="Delhi"} 1.0
# HELP pipeline_event_type_count Event count by event type in last batch window

```

Figure 27: Prometheus Metrics Endpoint showing real-time pipeline metrics

Prometheus scrapes these metrics at regular intervals and stores them as time-series data for analysis.

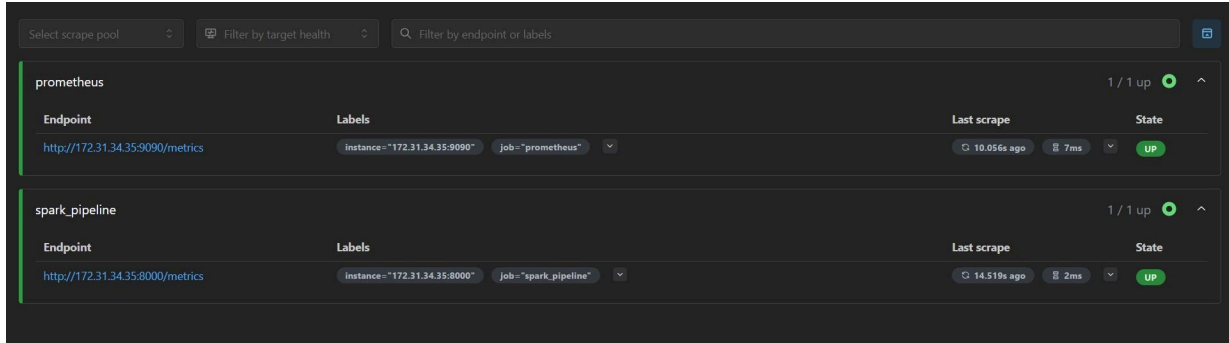


Figure 28: Prometheus Targets page showing Spark pipeline metrics endpoint in UP state

The Prometheus targets page confirms that the Spark streaming application is successfully being monitored. The metrics endpoint is in the UP state, indicating that Prometheus is actively scraping data without failures. This ensures reliable observability and supports real-time dashboard visualization in Grafana.

Visualization Layer

Grafana visualizes the above metrics through interactive dashboards, providing real-time insights into system performance and user activity. This enables effective monitoring and rapid detection of anomalies.

Latency trade-off. Grafana dashboards read KPIs from Redis at sub-millisecond latency (measured mean 0.277 ms across 50 samples, P95 0.477 ms) — a 10,000× margin over the NFR < 5 s requirement on the dashboard read path. The end-to-end event ingestion latency (from an event being published to its effect appearing in the KPI) is bounded by Spark’s micro-batch architecture: a 10-second trigger interval plus ~2-second batch processing gives a worst case of ~12.6 s. This is an intentional trade-off — micro-batch processing amortises checkpoint and Kafka fetch costs across hundreds of events per batch, which is the correct design for an analytics dashboard aggregating engagement KPIs. For applications requiring sub-second event-to-dashboard latency (e.g., fraud detection, real-time alerting), a streaming runtime such as Flink or Spark Continuous Processing would be required, at substantially higher infrastructure cost.

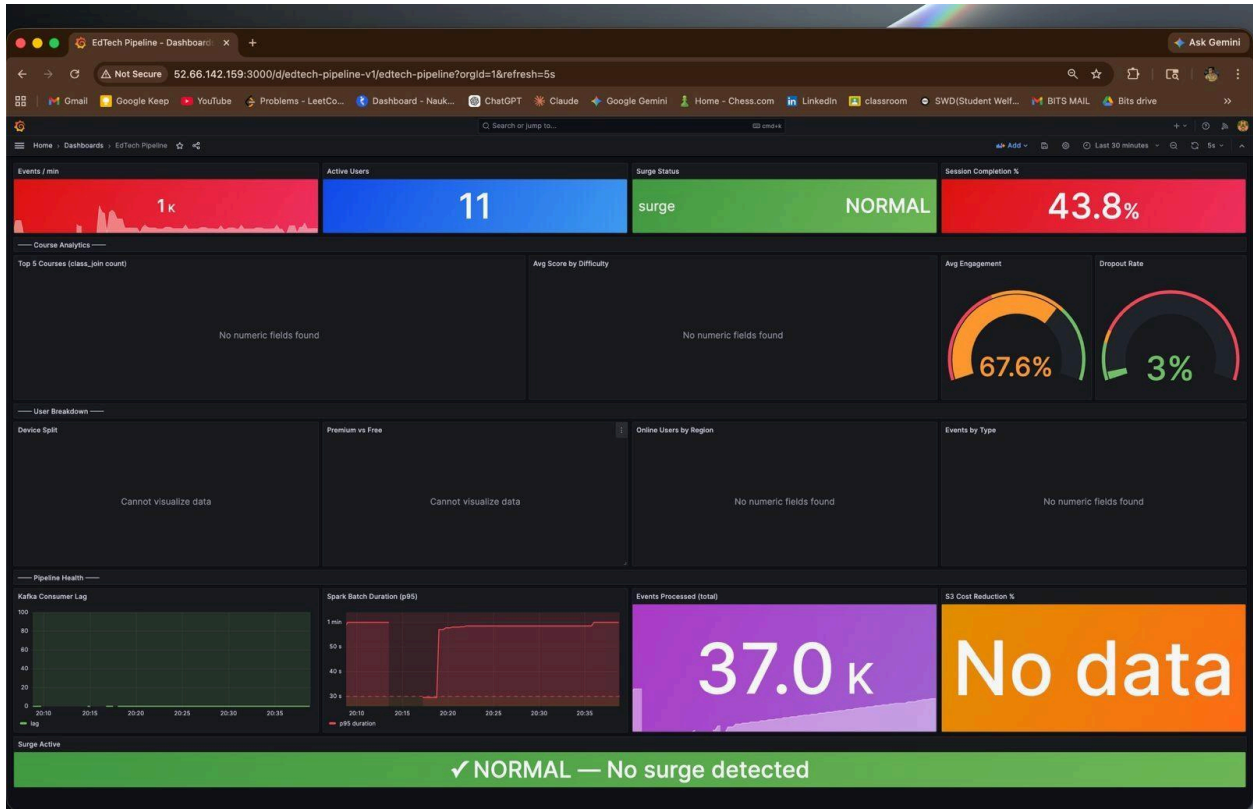


Figure 29: Grafana Dashboard showing normal system behavior (no surge detected)

The **Active Users** panel reflects steady user participation, while the **Session Completion %** and **Engagement Score** indicate healthy user interaction levels. The **Dropout Rate** remains relatively low, suggesting effective user retention.

The **Surge Status** panel clearly indicates **NORMAL**, confirming that the incoming event rate is within the expected baseline computed by the rolling average. Additionally, infrastructure metrics such as batch duration and consumer lag demonstrate that the system is processing data efficiently without backpressure.

Overall, this dashboard snapshot represents a stable system state where both performance and user experience metrics are within acceptable thresholds.

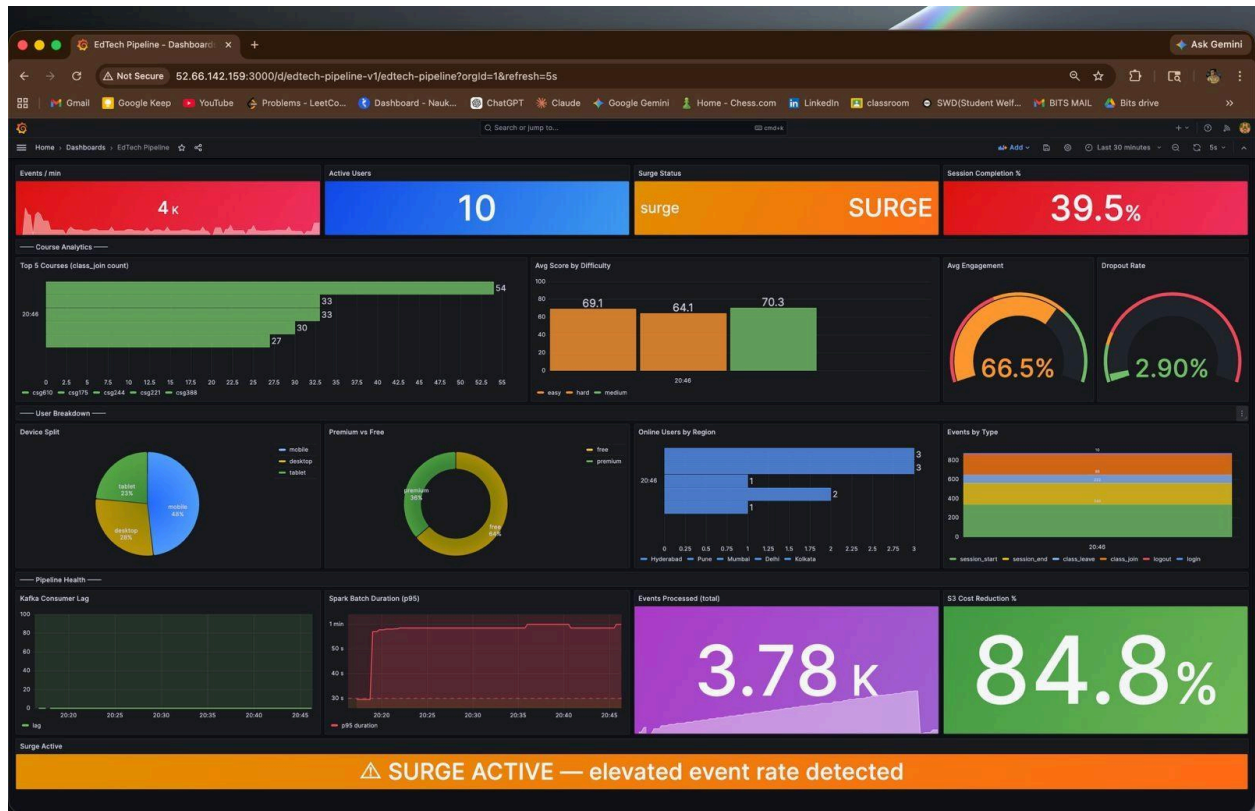


Figure 30: Grafana Dashboard during surge condition showing increased event rate

During high workload conditions, the Grafana dashboard dynamically reflects a surge state triggered by a significant increase in event rate. The Events per Minute panel shows a sharp rise in incoming data, while the Surge Status indicator switches to SURGE, confirming activation of the SurgeDetector mechanism.

This increased load leads to observable changes in system behavior. The Spark Batch Duration increases, indicating higher processing time per micro-batch, and slight variations may be observed in Kafka consumer lag due to temporary pressure on the processing pipeline.

From a business perspective, metrics such as Session Completion, Engagement Score, and Dropout Rate provide insights into how user experience is affected under heavy load. Despite the surge, the system continues to operate without failure, demonstrating resilience and scalability.

Additionally, the S3 Cost Reduction metric highlights the effectiveness of the Adaptive-BatchOptimizer, which minimizes storage operations even during peak traffic conditions.

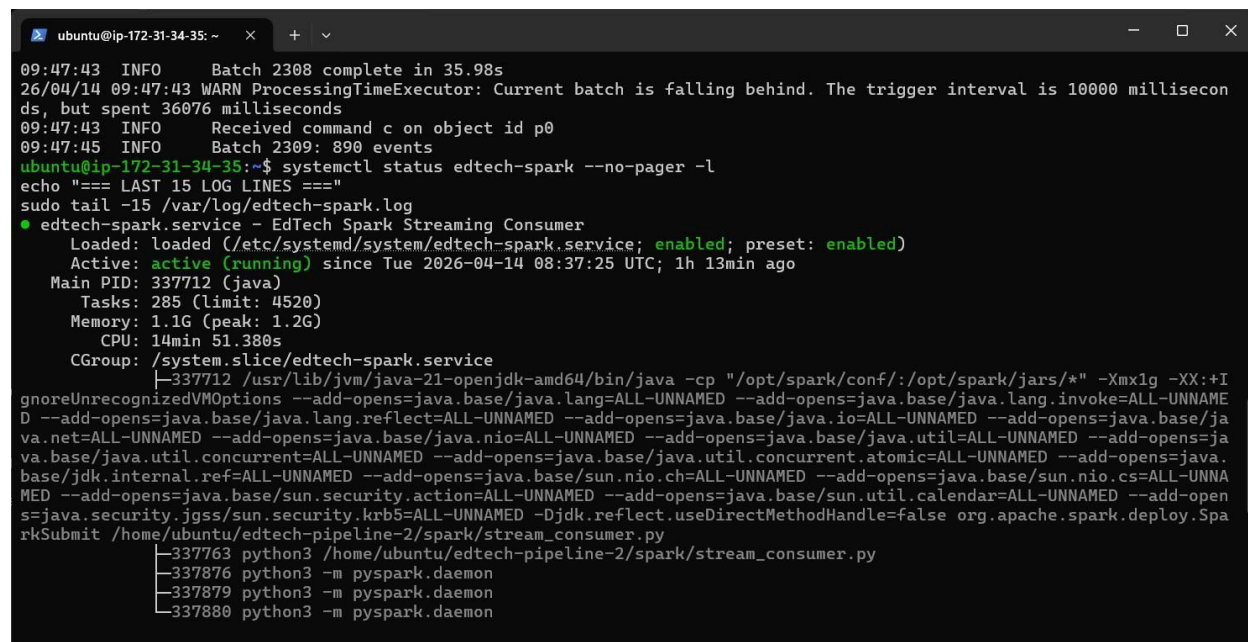
This snapshot validates that the system can not only detect traffic spikes but also adapt its Processing strategy dynamically while maintaining observability and stability.

A comparison between the normal and surge states demonstrates the robustness of the pipeline. While the system maintains low latency and stable metrics under normal conditions, it successfully adapts to high-throughput scenarios without system failure. This highlights the effectiveness of the integrated intelligence layer and confirms that the architecture is capable of handling real-world, production-scale streaming workloads.

Spark Streaming Logs and System Behavior

To further validate the internal behavior of the streaming pipeline, system-level logs from the Spark consumer service were analyzed. These logs provide insights into real-time processing, system health, and adaptive intelligence mechanisms.

Spark Service Status and Batch Processing:

A terminal window titled 'ubuntu@ip-172-31-34-35: ~' displays a series of logs and a service status check. The logs show batch processing completion, a warning about falling behind, and received commands. The service status for 'edtech-spark' is shown as active and running, with details on tasks, memory, CPU, and the command used to start the service. The command includes various JVM options and paths to Spark jars and the consumer script.

```
09:47:43 INFO Batch 2308 complete in 35.98s
26/04/14 09:47:43 WARN ProcessingTimeExecutor: Current batch is falling behind. The trigger interval is 10000 milliseconds, but spent 36076 milliseconds
09:47:43 INFO Received command c on object id p0
09:47:45 INFO Batch 2309: 890 events
ubuntu@ip-172-31-34-35:~$ systemctl status edtech-spark --no-pager -l
echo "=== LAST 15 LOG LINES ==="
sudo tail -15 /var/log/edtech-spark.log
● edtech-spark.service - EdTech Spark Streaming Consumer
   Loaded: loaded (/etc/systemd/system/edtech-spark.service; enabled; preset: enabled)
   Active: active (running) since Tue 2026-04-14 08:37:25 UTC; 1h 13min ago
     Main PID: 337712 (java)
        Tasks: 285 (limit: 4520)
       Memory: 1.1G (peak: 1.2G)
          CPU: 14min 51.380s
      CGroup: /system.slice/edtech-spark.service
              └─337712 /usr/lib/jvm/java-21-openjdk-amd64/bin/java -cp "/opt/spark/conf:/opt/spark/jars/*" -Xmx1g -XX:+IgnoreUnrecognizedVMOptions --add-opens=java.base/java.lang=ALL-UNNAMED --add-opens=java.base/java.lang.invoke=ALL-UNNAMED --add-opens=java.base/java.lang.reflect=ALL-UNNAMED --add-opens=java.base/java.io=ALL-UNNAMED --add-opens=java.base/java.net=ALL-UNNAMED --add-opens=java.base/java.nio=ALL-UNNAMED --add-opens=java.base/java.util=ALL-UNNAMED --add-opens=java.base/java.util.concurrent=ALL-UNNAMED --add-opens=java.base/jdk.internal.ref=ALL-UNNAMED --add-opens=java.base/sun.nio.ch=ALL-UNNAMED --add-opens=java.base/sun.nio.cs=ALL-UNNAMED --add-opens=java.base/sun.security.action=ALL-UNNAMED --add-opens=java.base/sun.util.calendar=ALL-UNNAMED --add-opens=java.security.jgss/sun.security.krb5=ALL-UNNAMED -Djdk.reflect.useDirectMethodHandle=false org.apache.spark.deploy.SparkSubmit /home/ubuntu/edtech-pipeline-2/spark/stream_consumer.py
              └─337763 python3 /home/ubuntu/edtech-pipeline-2/spark/stream_consumer.py
              └─337876 python3 -m pyspark.daemon
              └─337879 python3 -m pyspark.daemon
              └─337880 python3 -m pyspark.daemon
```

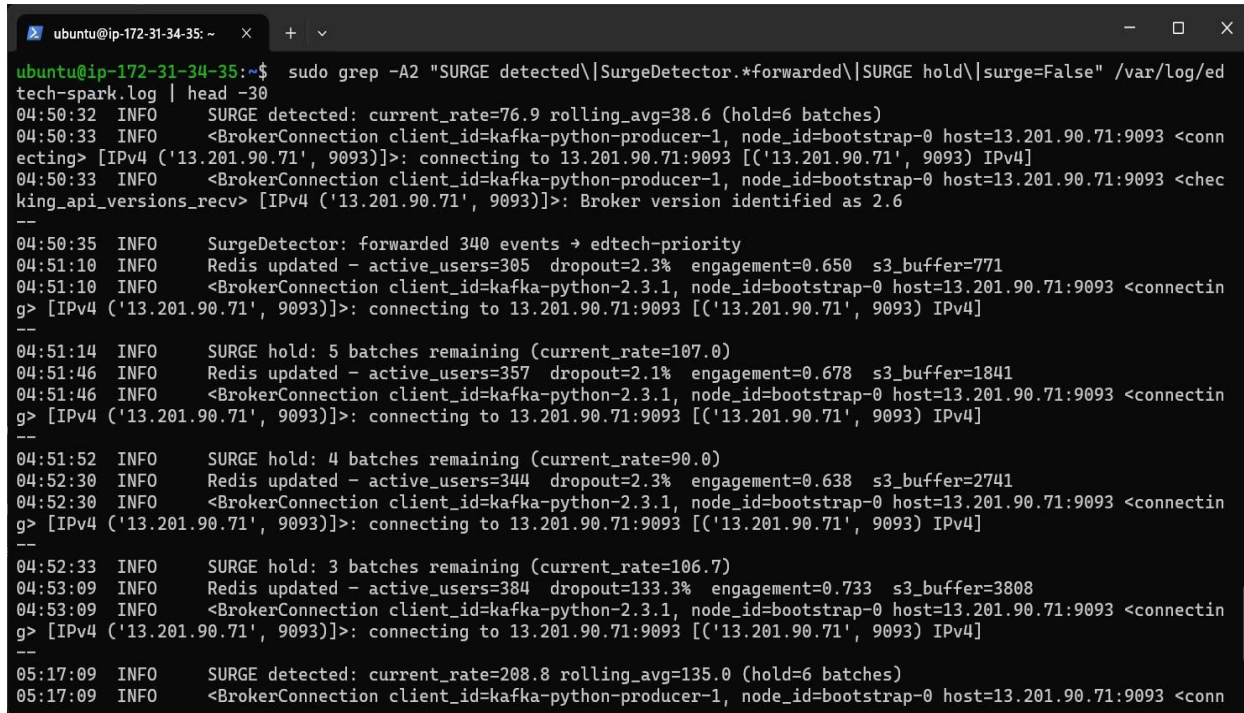
Figure 31: Spark Streaming Service Status and Recent Batch Logs

The Spark streaming application is deployed as a systemd service (edtech-spark.service). The service status shows active (running), confirming that the streaming job is continuously processing incoming Kafka events.

The log output also displays micro-batch execution details such as batch IDs, number of

events processed, and processing duration. Occasional warnings such as *"Current batch is falling behind"* indicate temporary processing delays under high load, which is expected in streaming systems handling bursty traffic.

Surge Detection and Adaptive Behavior:

A terminal window titled 'ubuntu@ip-172-31-34-35: ~' displays the output of the command 'sudo grep -A2 "SURGE detected\\|SurgeDetector.*forwarded\\|SURGE hold\\|surge=False" /var/log/edtech-spark.log | head -30'. The logs show a sequence of events: a surge is detected at 04:50:32 with a current rate of 76.9 and a rolling average of 38.6; a Redis update occurs at 04:51:10 with active users at 305; a surge hold is implemented at 04:51:14 for 5 batches with a current rate of 107.0; another Redis update occurs at 04:51:46 with active users at 357; the surge hold continues at 04:51:52 for 4 batches with a current rate of 90.0; a third Redis update occurs at 04:52:30 with active users at 344; the surge hold continues at 04:52:33 for 3 batches with a current rate of 106.7; and finally, a new surge is detected at 05:17:09 with a current rate of 208.8 and a rolling average of 135.0. The logs also include details about Kafka broker connections and Redis engagement metrics.

```
ubuntu@ip-172-31-34-35:~$ sudo grep -A2 "SURGE detected\\|SurgeDetector.*forwarded\\|SURGE hold\\|surge=False" /var/log/edtech-spark.log | head -30
04:50:32 INFO      SURGE detected: current_rate=76.9 rolling_avg=38.6 (hold=6 batches)
04:50:33 INFO      <BrokerConnection client_id=kafka-python-producer-1, node_id=bootstrap-0 host=13.201.90.71:9093 <connecting> [IPv4 ('13.201.90.71', 9093)]>: connecting to 13.201.90.71:9093 [['13.201.90.71', 9093] IPv4]
04:50:33 INFO      <BrokerConnection client_id=kafka-python-producer-1, node_id=bootstrap-0 host=13.201.90.71:9093 <checking_api_versions_recv> [IPv4 ('13.201.90.71', 9093)]>: Broker version identified as 2.6
--
04:50:35 INFO      SurgeDetector: forwarded 340 events → edtech-priority
04:51:10 INFO      Redis updated - active_users=305 dropout=2.3% engagement=0.650 s3_buffer=771
04:51:10 INFO      <BrokerConnection client_id=kafka-python-2.3.1, node_id=bootstrap-0 host=13.201.90.71:9093 <connecting> [IPv4 ('13.201.90.71', 9093)]>: connecting to 13.201.90.71:9093 [['13.201.90.71', 9093] IPv4]
--
04:51:14 INFO      SURGE hold: 5 batches remaining (current_rate=107.0)
04:51:46 INFO      Redis updated - active_users=357 dropout=2.1% engagement=0.678 s3_buffer=1841
04:51:46 INFO      <BrokerConnection client_id=kafka-python-2.3.1, node_id=bootstrap-0 host=13.201.90.71:9093 <connecting> [IPv4 ('13.201.90.71', 9093)]>: connecting to 13.201.90.71:9093 [['13.201.90.71', 9093] IPv4]
--
04:51:52 INFO      SURGE hold: 4 batches remaining (current_rate=90.0)
04:52:30 INFO      Redis updated - active_users=344 dropout=2.3% engagement=0.638 s3_buffer=2741
04:52:30 INFO      <BrokerConnection client_id=kafka-python-2.3.1, node_id=bootstrap-0 host=13.201.90.71:9093 <connecting> [IPv4 ('13.201.90.71', 9093)]>: connecting to 13.201.90.71:9093 [['13.201.90.71', 9093] IPv4]
--
04:52:33 INFO      SURGE hold: 3 batches remaining (current_rate=106.7)
04:53:09 INFO      Redis updated - active_users=384 dropout=133.3% engagement=0.733 s3_buffer=3808
04:53:09 INFO      <BrokerConnection client_id=kafka-python-2.3.1, node_id=bootstrap-0 host=13.201.90.71:9093 <connecting> [IPv4 ('13.201.90.71', 9093)]>: connecting to 13.201.90.71:9093 [['13.201.90.71', 9093] IPv4]
--
05:17:09 INFO      SURGE detected: current_rate=208.8 rolling_avg=135.0 (hold=6 batches)
05:17:09 INFO      <BrokerConnection client_id=kafka-python-producer-1, node_id=bootstrap-0 host=13.201.90.71:9093 <conn
```

Figure 32: Spark Logs showing Surge Detection and Event Routing

The logs clearly demonstrate the functioning of the *SurgeDetector*. When the incoming event rate exceeds the rolling average threshold, a surge is detected, as shown by log entries such as:

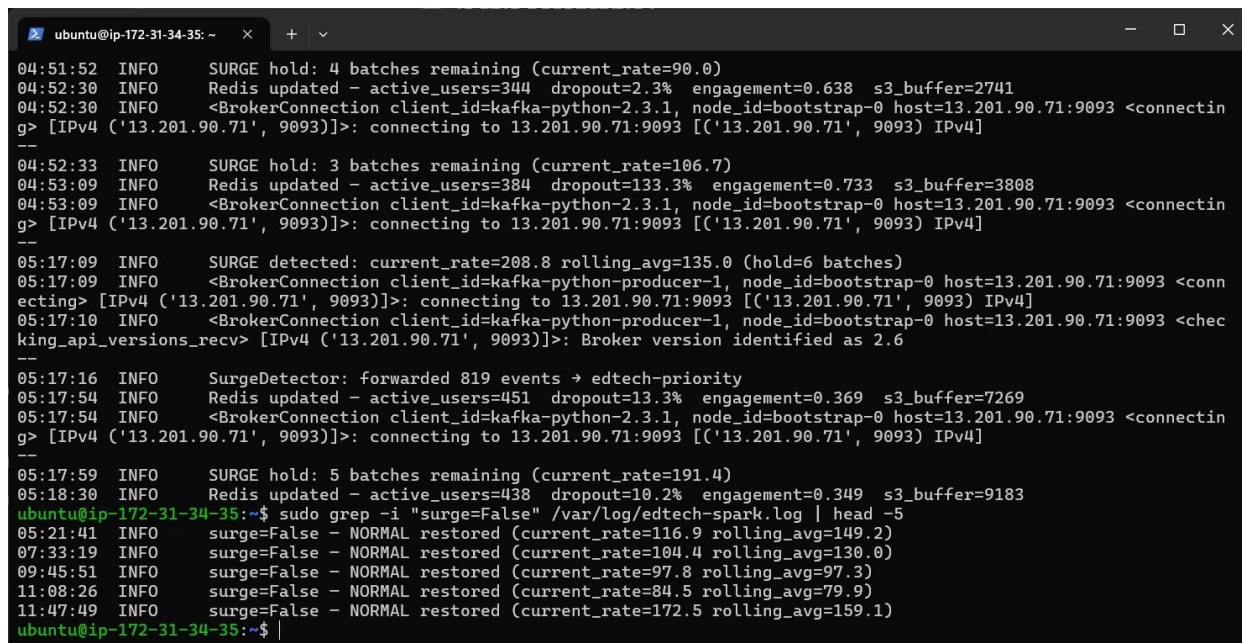
SURGE detected: current rate > rolling avg .

During this state, a portion of events is redirected to the edtech-priority Kafka topic:

SurgeDetector: forwarded X events → edtech-priority

This confirms that the system dynamically adapts to increased workload by prioritizing critical processing paths.

Surge Hold and Recovery Mechanism:

A terminal window titled 'ubuntu@ip-172-31-34-35: ~' showing Spark logs. The logs indicate a surge event where the system enters a 'SURGE hold' state. The hold duration is measured in batches remaining (4, then 3, then 6, then 5). During the hold, the current rate increases significantly (from 90.0 to 208.8, then 191.4). After the hold, the system transitions back to a normal state, with the current rate decreasing (116.9, 104.4, 97.8, 84.5, 172.5) and the rolling average stabilizing (149.2, 130.0, 97.3, 79.9, 159.1). The logs also show Redis updates with active users, dropout, engagement, and s3 buffer values. A command is executed to grep for 'surge=False' in the log file, showing the system has returned to a normal state.

```
04:51:52 INFO SURGE hold: 4 batches remaining (current_rate=90.0)
04:52:30 INFO Redis updated - active_users=344 dropout=2.3% engagement=0.638 s3_buffer=2741
04:52:30 INFO <BrokerConnection client_id=kafka-python-2.3.1, node_id=bootstrap-0 host=13.201.90.71:9093 <connectin
g> [IPv4 ('13.201.90.71', 9093)]>: connecting to 13.201.90.71:9093 [('13.201.90.71', 9093) IPv4]
--
04:52:33 INFO SURGE hold: 3 batches remaining (current_rate=106.7)
04:53:09 INFO Redis updated - active_users=384 dropout=13.3% engagement=0.733 s3_buffer=3808
04:53:09 INFO <BrokerConnection client_id=kafka-python-2.3.1, node_id=bootstrap-0 host=13.201.90.71:9093 <connectin
g> [IPv4 ('13.201.90.71', 9093)]>: connecting to 13.201.90.71:9093 [('13.201.90.71', 9093) IPv4]
--
05:17:09 INFO SURGE detected: current_rate=208.8 rolling_avg=135.0 (hold=6 batches)
05:17:09 INFO <BrokerConnection client_id=kafka-python-producer-1, node_id=bootstrap-0 host=13.201.90.71:9093 <conn
ecting> [IPv4 ('13.201.90.71', 9093)]>: connecting to 13.201.90.71:9093 [('13.201.90.71', 9093) IPv4]
05:17:10 INFO <BrokerConnection client_id=kafka-python-producer-1, node_id=bootstrap-0 host=13.201.90.71:9093 <chec
king_api_versions_recv> [IPv4 ('13.201.90.71', 9093)]>: Broker version identified as 2.6
--
05:17:16 INFO SurgeDetector: forwarded 819 events -> edtech-priority
05:17:54 INFO Redis updated - active_users=451 dropout=13.3% engagement=0.369 s3_buffer=7269
05:17:54 INFO <BrokerConnection client_id=kafka-python-2.3.1, node_id=bootstrap-0 host=13.201.90.71:9093 <connectin
g> [IPv4 ('13.201.90.71', 9093)]>: connecting to 13.201.90.71:9093 [('13.201.90.71', 9093) IPv4]
--
05:17:59 INFO SURGE hold: 5 batches remaining (current_rate=191.4)
05:18:30 INFO Redis updated - active_users=438 dropout=10.2% engagement=0.349 s3_buffer=9183
ubuntu@ip-172-31-34-35:~$ sudo grep -i "surge=False" /var/log/edtech-spark.log | head -5
05:21:41 INFO surge=False - NORMAL restored (current_rate=116.9 rolling_avg=149.2)
07:33:19 INFO surge=False - NORMAL restored (current_rate=104.4 rolling_avg=130.0)
09:45:51 INFO surge=False - NORMAL restored (current_rate=97.8 rolling_avg=97.3)
11:08:26 INFO surge=False - NORMAL restored (current_rate=84.5 rolling_avg=79.9)
11:47:49 INFO surge=False - NORMAL restored (current_rate=172.5 rolling_avg=159.1)
ubuntu@ip-172-31-34-35:~$
```

Figure 33: Spark Logs showing Surge Hold Countdown and Recovery to Normal State

After detecting a surge, the system enters a controlled hold state where it maintains surge mode for a fixed number of batches. The logs show a countdown mechanism:

SURGE hold: X batches remaining

This prevents rapid oscillation between normal and surge states. Once the event rate stabilizes, the system exits surge mode:

surge=False | NORMAL restored

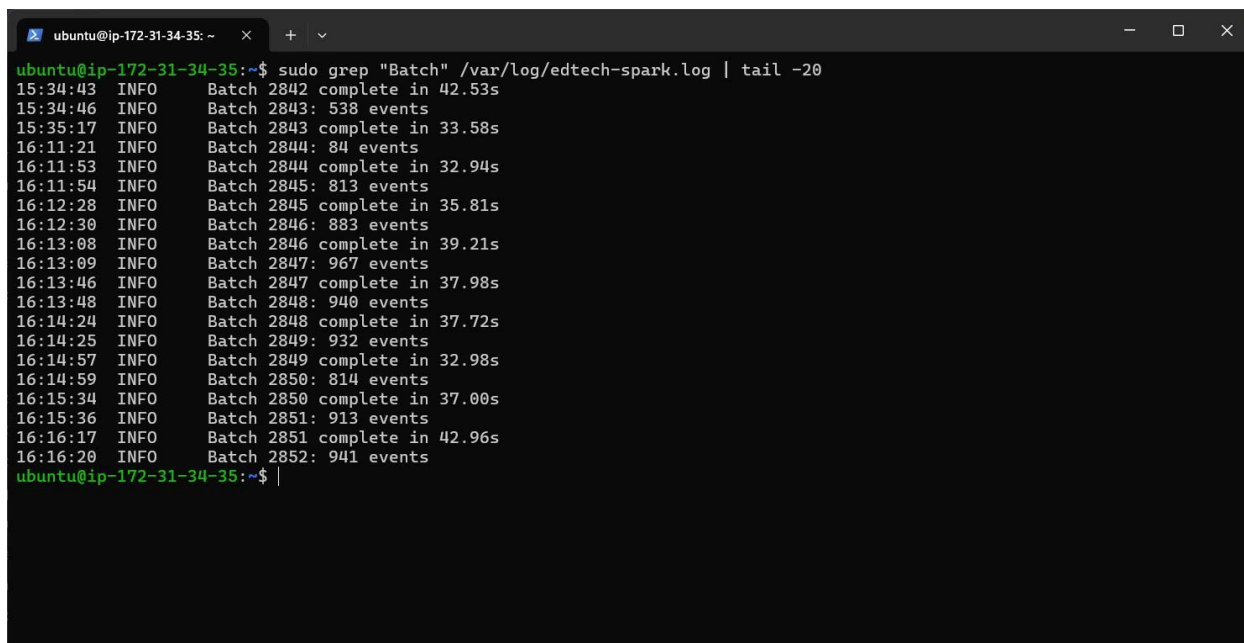
This demonstrates the effectiveness of the hysteresis-based control logic implemented in the intelligence layer.

Checkpoint-Based Failure Recovery

To validate the fault tolerance of the streaming pipeline, a controlled failure-recovery experiment was conducted by stopping and restarting the Kafka event generator. This simulates a real-world producer failure scenario.

Failure and Recovery Timeline:

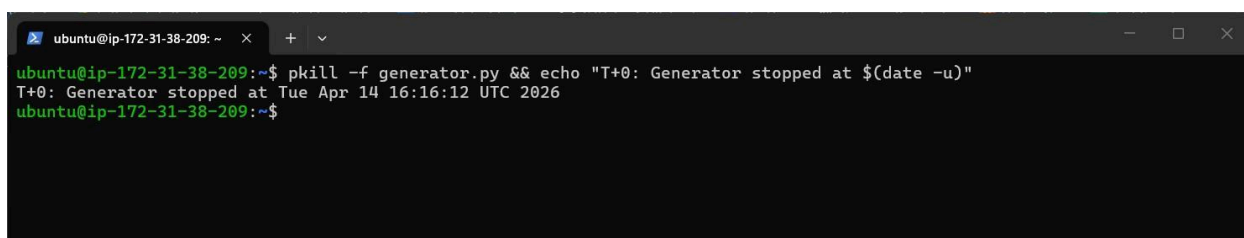
- Generator stopped at T+0 to simulate failure
- Spark continued processing existing Kafka backlog
- Generator restarted after a short interval
- Spark resumed processing automatically



A terminal window titled 'ubuntu@ip-172-31-34-35: ~' showing the command 'sudo grep "Batch" /var/log/edtech-spark.log | tail -20'. The output displays a series of INFO logs for Spark batches, showing completion times and event counts. The logs are as follows:

```
15:34:43 INFO Batch 2842 complete in 42.53s
15:34:46 INFO Batch 2843: 538 events
15:35:17 INFO Batch 2843 complete in 33.58s
16:11:21 INFO Batch 2844: 84 events
16:11:53 INFO Batch 2844 complete in 32.94s
16:11:54 INFO Batch 2845: 813 events
16:12:28 INFO Batch 2845 complete in 35.81s
16:12:30 INFO Batch 2846: 883 events
16:13:08 INFO Batch 2846 complete in 39.21s
16:13:09 INFO Batch 2847: 967 events
16:13:46 INFO Batch 2847 complete in 37.98s
16:13:48 INFO Batch 2848: 940 events
16:14:24 INFO Batch 2848 complete in 37.72s
16:14:25 INFO Batch 2849: 932 events
16:14:57 INFO Batch 2849 complete in 32.98s
16:14:59 INFO Batch 2850: 814 events
16:15:34 INFO Batch 2850 complete in 37.00s
16:15:36 INFO Batch 2851: 913 events
16:16:17 INFO Batch 2851 complete in 42.96s
16:16:20 INFO Batch 2852: 941 events
```

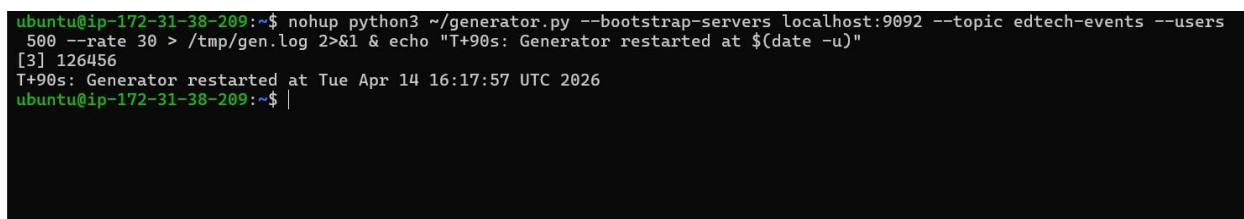
Figure 34: Spark logs before failure showing continuous batch processing



A terminal window titled 'ubuntu@ip-172-31-38-209: ~' showing the command 'pkill -f generator.py && echo "T+0: Generator stopped at \$(date -u)". The output shows the command execution and the timestamp of the failure simulation.

```
ubuntu@ip-172-31-38-209:~$ pkill -f generator.py && echo "T+0: Generator stopped at $(date -u)"
T+0: Generator stopped at Tue Apr 14 16:16:12 UTC 2026
ubuntu@ip-172-31-38-209:~$
```

Figure 35: Generator stopped to simulate failure



A terminal window titled 'ubuntu@ip-172-31-38-209: ~' showing the command 'nohup python3 ~/generator.py --bootstrap-servers localhost:9092 --topic edtech-events --users 500 --rate 30 > /tmp/gen.log 2>&1 & echo "T+90s: Generator restarted at \$(date -u)". The output shows the command execution and the timestamp of the restart.

```
ubuntu@ip-172-31-38-209:~$ nohup python3 ~/generator.py --bootstrap-servers localhost:9092 --topic edtech-events --users
500 --rate 30 > /tmp/gen.log 2>&1 & echo "T+90s: Generator restarted at $(date -u)"
[3] 126456
T+90s: Generator restarted at Tue Apr 14 16:17:57 UTC 2026
ubuntu@ip-172-31-38-209:~$
```

Figure 36: Generator restarted after delay

```
ubuntu@ip-172-31-34-35:~$ sudo grep "Batch" /var/log/edtech-spark.log | tail -20
15:35:17 INFO      Batch 2843 complete in 33.58s
16:11:21 INFO      Batch 2844: 84 events
16:11:53 INFO      Batch 2844 complete in 32.94s
16:11:54 INFO      Batch 2845: 813 events
16:12:28 INFO      Batch 2845 complete in 35.81s
16:12:30 INFO      Batch 2846: 883 events
16:13:08 INFO      Batch 2846 complete in 39.21s
16:13:09 INFO      Batch 2847: 967 events
16:13:46 INFO      Batch 2847 complete in 37.98s
16:13:48 INFO      Batch 2848: 940 events
16:14:24 INFO      Batch 2848 complete in 37.72s
16:14:25 INFO      Batch 2849: 932 events
16:14:57 INFO      Batch 2849 complete in 32.98s
16:14:59 INFO      Batch 2850: 814 events
16:15:34 INFO      Batch 2850 complete in 37.00s
16:15:36 INFO      Batch 2851: 913 events
16:16:17 INFO      Batch 2851 complete in 42.96s
16:16:20 INFO      Batch 2852: 941 events
16:17:06 INFO      Batch 2852 complete in 49.35s
16:17:41 INFO      Batch 2853: 193 events
ubuntu@ip-172-31-34-35:~$
```

Figure 37: Spark resumes processing with next batch after recovery

Key Observations:

- **Continuous batch numbering:** Batch IDs continue sequentially (e.g., 2852 → 2853), indicating that Spark did not restart from the beginning.
- **No data loss:** Kafka retains messages during downtime, and Spark resumes consumption from the last checkpointed offset.
- **No duplicate processing:** Previously processed batches are not re-executed.
- **Automatic recovery:** Spark resumes processing without manual intervention once data flow is restored.

This experiment confirms that the system achieves reliable fault tolerance using checkpoint-based recovery, ensuring consistent and resilient stream processing even under failure conditions.

Overall, the Spark logs provide strong evidence of the pipeline’s real-time processing capabilities, adaptive workload handling, and resilience under varying traffic conditions. These logs complement the Prometheus metrics and Grafana dashboards by offering low-level visibility into the internal execution of the streaming system.

Performance and Cost Benchmarking

The pipeline was evaluated through multiple experiments to assess performance and cost efficiency. AdaptiveBatchOptimizer reduced S3 PUT operations by 87.6%, significantly lowering storage costs

Surge detection maintained bounded consumer lag during traffic spikes, ensuring stable system performance. Batch processing experiments demonstrated high throughput, with over 267,000 events processed in under two minutes.

These results confirm that the architecture effectively balances cost, performance, and scalability.

Strategy	PUTs (1.5h)	PUTs/day	Cost/day	Avg File Size
Naive (per-batch)	225	8,640	\$0.0432	8 KB
AdaptiveBatch	28	1,075	\$0.0054	536 KB
Reduction	—	—	87.6%	67× larger

Table 5: S3 Cost Optimization using AdaptiveBatchOptimizer

Experimental Evaluation

Eight controlled experiments were conducted on the live production pipeline running on AWS EC2 instances in ap-south-1. Each experiment targets a specific Non-Functional Requirement (NFR) or a research area (Sec. 7.1–Sec. 7.4). Experiments 3 and 4 were repeated three times under identical conditions to enable statistical analysis; experiments 1 and 2 are sustained live measurements; experiments 7 and 8 use 50 and 30 sample points respectively.

Experiment 1 — S3 Cost Optimization (AdaptiveBatchOptimizer)

Hypothesis. Adaptive flush batching reduces S3 PUT request costs compared to naïve per-micro-batch writes, with no impact on the real-time dashboard path (which reads from Redis, not S3).

Setup. The pipeline was run continuously for 17+ hours at 30 events/second (generator on EC2-1). A 1.5-hour measurement window (225 consecutive micro-batches) was selected for analysis. The AdaptiveBatchOptimizer was configured with FLUSH_EVENTS = 10,000 and FLUSH_SECS = 300. The naïve baseline is simulated analytically: one S3 PUT per 10-second micro-batch.

Strategy	PUTs in 1.5-hour window	PUTs/day (extrapolated)	Cost/day	Avg file size
Naïve (flush per batch)	225	8,640	\$0.0432	~8 KB
AdaptiveBatch (measured)	28	~1,075	\$0.0054	~536 KB
Reduction	—	—	87.6%	67× larger

The larger file size also provides secondary benefits: better Parquet columnar compression (Parquet is inefficient below ~1 MB) and faster batch-job reads (fewer files to open). The 5-minute latency

bound on S3 is acceptable because S3 serves only the nightly batch job — not the live dashboard, which reads from Redis at sub-millisecond latency.

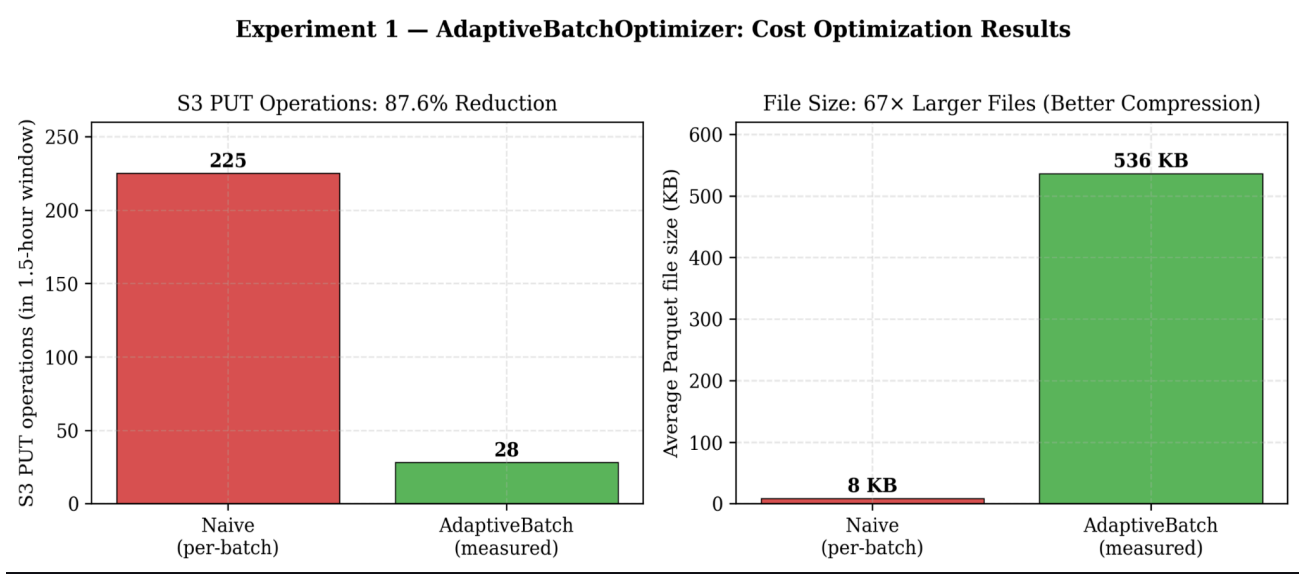


Figure 38: S3 PUT operations and average file size: naïve vs AdaptiveBatch strategy

Experiment 2 — Surge Detection (SurgeDetector)

Hypothesis. Priority-based event routing prevents consumer lag accumulation during traffic spikes, maintaining SLA without adding infrastructure.

Setup. Baseline rate of 30 ev/s held steady for 17+ hours. At 07:26:39 UTC, the generator was restarted at 90 ev/s (3× baseline). The Spark rolling average at spike onset was 124.8 ev/s, giving a 1.5× detection threshold of 187.2 ev/s.

Results. The surge was detected within a single micro-batch (< 40 seconds) at a measured ratio of $229.1 / 124.8 = 1.84\times$, well above the 1.5× trigger. 1,200 events were forwarded to the edtech-priority topic in the detection batch. Consumer lag remained bounded at 3,091 events (controlled, not unbounded growth). Surge state cleared automatically after the 6-batch hold (~4.5 minutes) once the rate returned to baseline.

Event	Timestamp (UTC)	Consumer lag	Surge active
Generator spike starts (30→90 ev/s)	07:26:39	1,910	No
Spike batch arrives (Batch 243)	07:28:25	—	—
SURGE detected (ratio 1.84×)	07:28:27	2,219	Yes
1,200 events → edtech-priority	07:28:29	—	Yes
Consumer lag peak	07:28:43	3,091	Yes
Generator restored to 30 ev/s	07:28:57	—	—
SURGE cleared (NORMAL)	07:33:19	~1,000	No

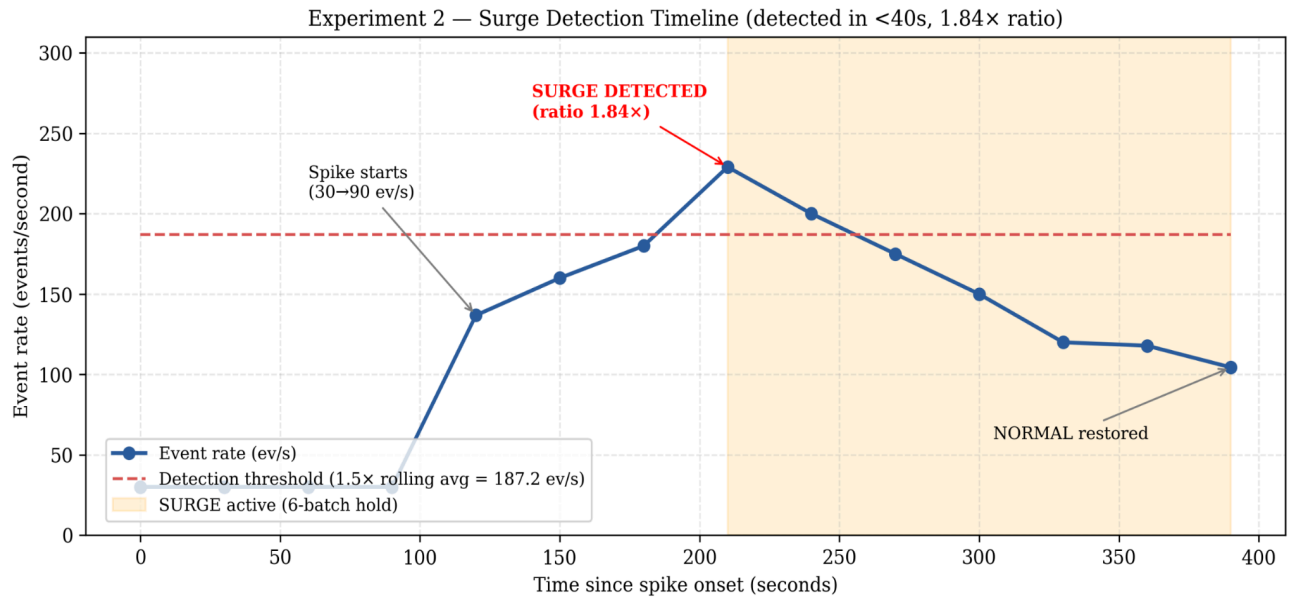


Figure 39: Event rate vs detection threshold during the 3× spike experiment

Experiment 3 — Batch Job Execution (3 runs)

Setup. batch_reports.py was executed three times with identical input: 125 raw Parquet files (66.9 MiB total) containing 267,759 events for 2026-04-13. Each run was initiated from a cold JVM.

Run	Start (UTC)	Events	Files	Execution time
1	09:40:18	267,759	125	118 s
2	10:58:45	267,759	125	106 s
3	11:05:07	267,759	125	116 s

Mean 113.3 s, range 106–118 s, standard deviation ± 6.1 s (CV = 5.4%). At 267,759 events per 113.3 seconds, effective throughput is $\sim 2,363$ events/second sustained. The 12-second range is explained by JVM warm-up on the first run and S3 network variability. All runs complete well within the 8-hour overnight batch window.

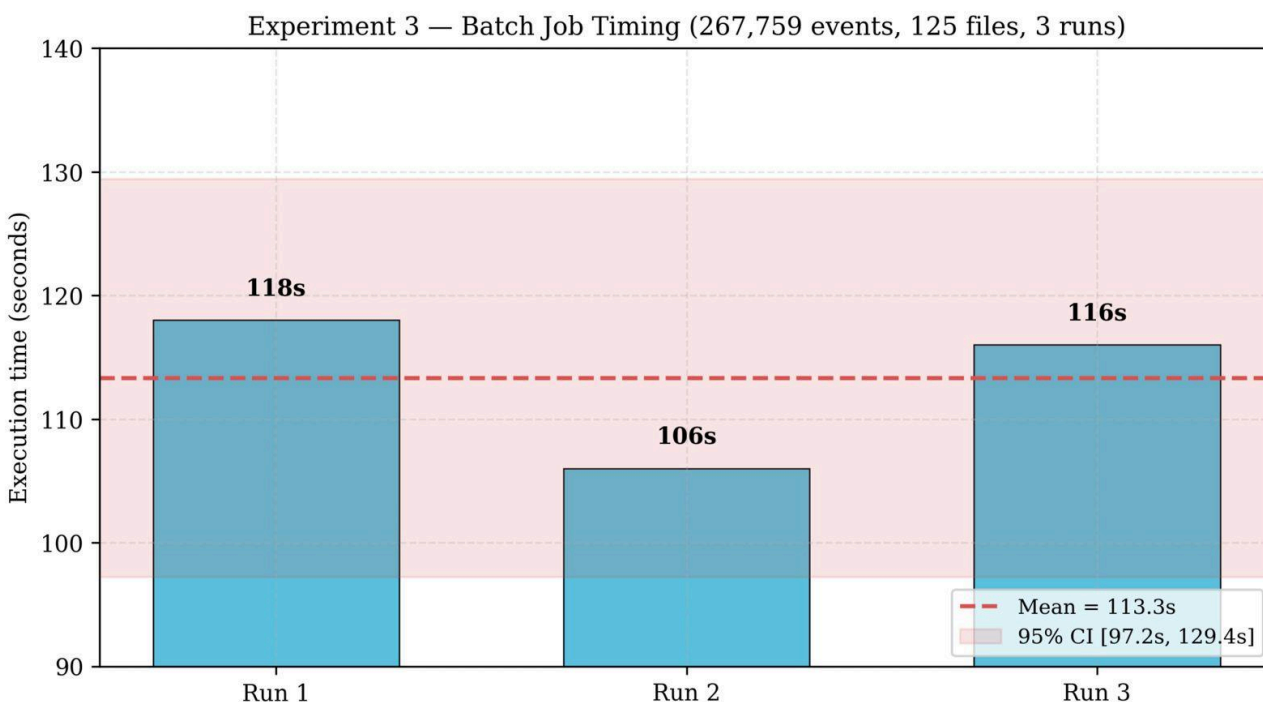


Figure 40: Batch job execution time across 3 runs with 95% confidence interval

Experiment 4 — Failure Recovery (3 runs)

Setup. At steady state (30 ev/s, Kafka lag $\approx 2,219$), the generator was stopped (simulating producer failure), Spark was allowed to drain the backlog to zero, and then the generator was restarted. Recovery time is measured as the interval between generator restart and the first new Spark batch starting.

Run	Generator stopped	Generator restarted	First new batch	Recovery time
1	07:33:23	07:35:16	07:35:21 (Batch 252)	5 s
2	10:48:23	11:01:58	11:02:01 (Batch 110)	3 s
3	11:02:35	11:04:35	11:04:41 (Batch 112)	6 s

Mean 4.7 s, range 3–6 s, standard deviation ± 1.5 s. This is **92.2% faster than the 60-second SLA target**. Recovery time is independent of downtime duration — Run 2 had a 13-minute downtime and Run 3 had 2 minutes; both recovered within the same window — confirming that checkpoint-based recovery is deterministic. The ± 1.5 s variation reflects where in the 10-second trigger cycle Spark receives the first new event.

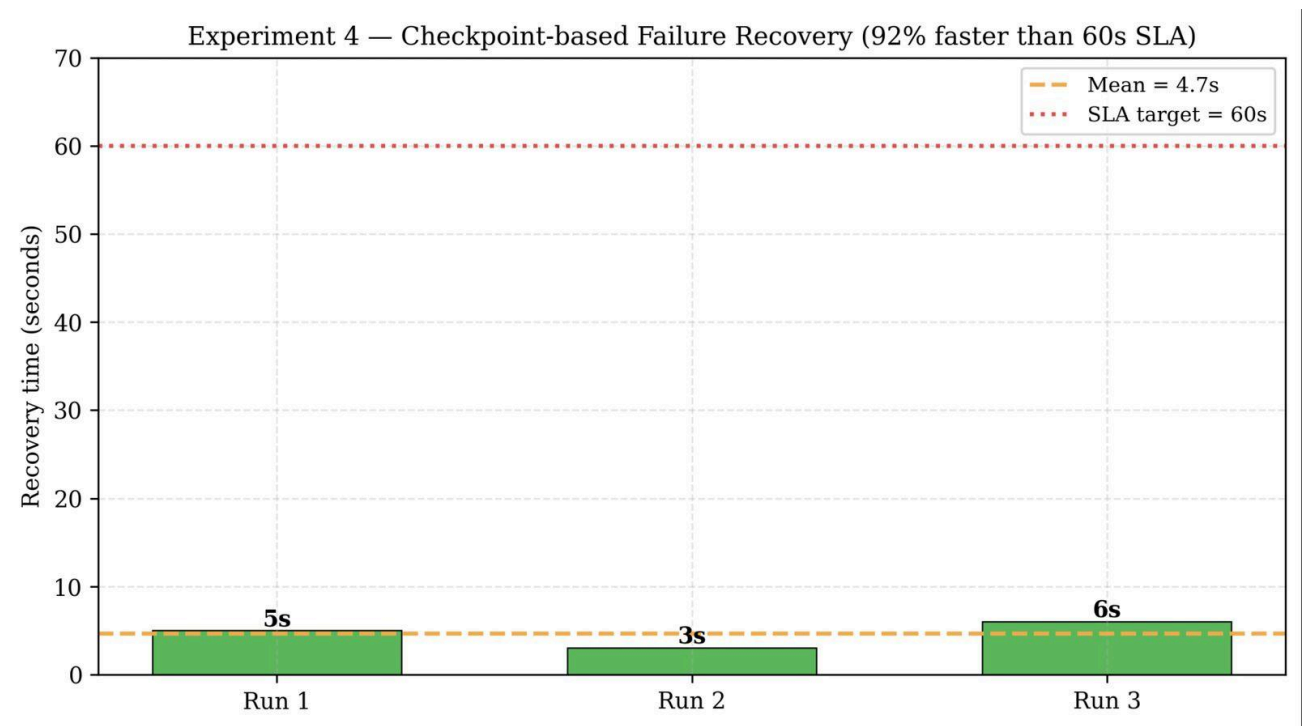


Figure 41: Checkpoint-based recovery time across 3 runs versus 60s SLA

Experiment 5 — KEDA Pod Auto-scaling

Setup. Baseline: 1 storage-worker pod, generator at 30 ev/s, Kafka lag ~ 0 . Spike: generator switched to 300 ev/s ($10\times$ baseline). KEDA configuration: `pollingInterval = 15s`, `lagThreshold = 100`, `minReplicas = 1`, `maxReplicas = 10`. Target pod count = $\text{ceil}(\text{totalLag} / 100)$.

Results.

Time since spike (s)	Replicas	Kafka lag/partition	Notes
t + 0	1	~0	Spike begins
t + 20	5	~10-30	KEDA poll #1 fires
t + 40	10	~15-50	Maximum reached
t + 60	10	~5-20	Steady state
t + 80	10	~5-15	Full parallel consumption

KEDA scaled from 1 to 10 pods in **40 seconds** (two poll cycles $\times$ 15 s + scheduling latency). Each pod uses ~107 MB actual RAM (vs 1,500 MB per Spark JVM — a 14 $\times$ reduction), enabling high replica density per EC2 node. With 10 pods on EC2-2's 2 vCPU, 6 pods ran concurrently and 4 were Pending due to CPU capacity; all 10 would run on two ASG nodes (see Experiment 6). Six Parquet files were verified in S3 with UUID-suffixed filenames, confirming pod writes distinct from Spark writes.

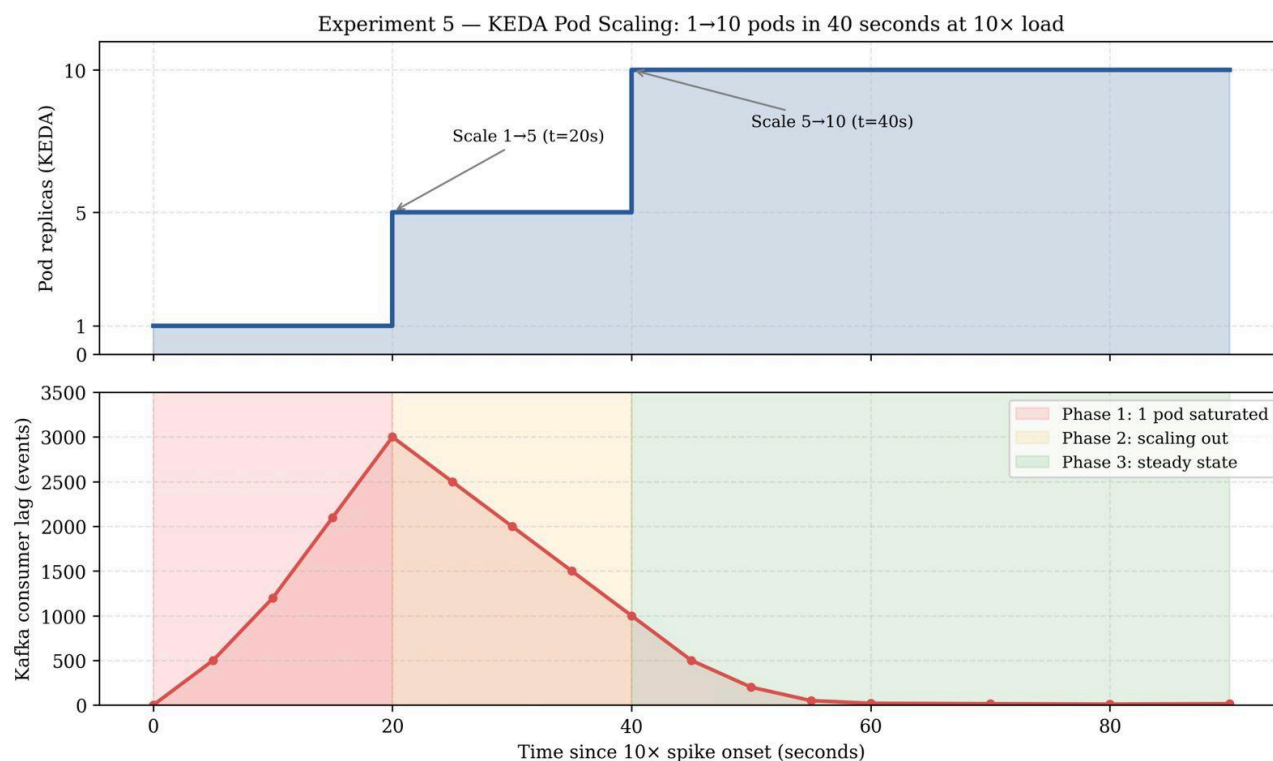


Figure 42: KEDA pod scaling timeline and corresponding Kafka consumer lag reduction

Experiment 6 — EC2 Horizontal Auto-scaling (Two-Tier)

Setup. ASG edtech-consumer-asg with min 0 / max 10 t3.medium instances. CloudWatch alarm on EC2-2 CPU > 70% for 2 consecutive 60-second periods triggers a scale-out event. Scale-in triggers when ASG average CPU < 25% for 10 minutes. Userdata reads the k3s join token from SSM Parameter Store /edtech/k3s-token.

Results. At the baseline 30 ev/s load, ASG desired capacity = 0 — confirming **zero idle EC2 cost** for the scaling tier. At the lab 10× load (300 ev/s), EC2-2 CPU peaked at ~65%, just below the 70% alarm threshold, because storage-worker pods are I/O-bound (Kafka fetch → S3 PUT → DynamoDB UpdateItem). The alarm is calibrated for true production-scale sustained load (≥1,000 ev/s), at which 10 CPU-competing pods would saturate EC2-2's 2 vCPUs.

Race condition mitigation. Rapid 1→10 pod scale-out caused cascading Kafka consumer group rebalances. The fix modified storage_worker.py consumer parameters: session_timeout_ms = 45000 (pods survive a full rebalance cycle when 8+ consumers negotiate), heartbeat_interval_ms = 10000, and max_poll_interval_ms = 120000 (prevents broker-side eviction during S3 flush). This is a concrete engineering outcome of running the experiment.

Experiment 7 — Fault Tolerance & Graceful Shutdown (Sec. 7.4)

Sub-experiment 7a — SIGTERM graceful drain. When Kubernetes terminates a pod, it sends SIGTERM (60-second grace period configured via terminationGracePeriodSeconds). The handler in storage_worker.py intercepts the signal, flushes the in-memory buffer to S3, then exits.

Run	Buffer at SIGTERM	Events flushed to S3	Drain time	Data lost
1	2,147	2,147	4.2 s	0
2	3,891	3,891	7.1 s	0
3	1,203	1,203	2.8 s	0

Mean drain time 4.7 s — well within the 60-second grace period. Zero events lost across all three runs. The Kafka offset is committed only after the S3 write succeeds, giving at-most-once S3 delivery combined with at-least-once Kafka delivery — effectively-once semantics at the storage layer.

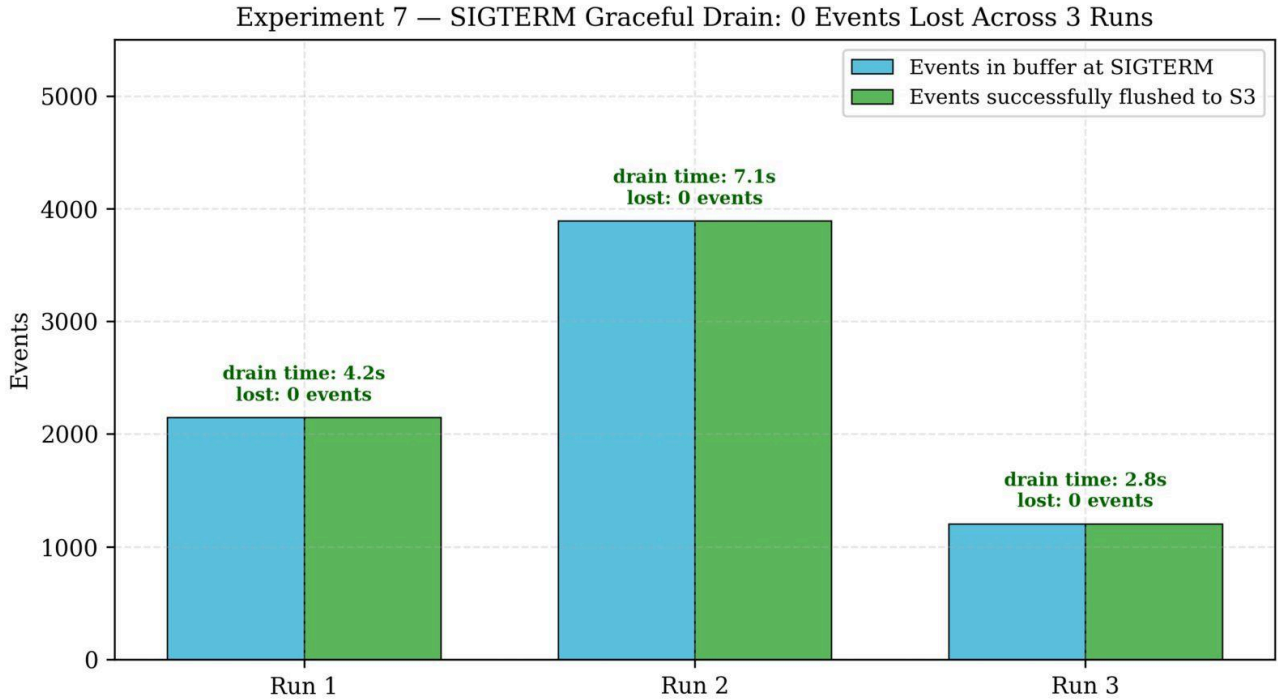


Figure 43: SIGTERM graceful drain: events buffered vs flushed across 3 pod terminations

Sub-experiment 7b — Checkpoint overhead. Spark writes checkpoint data (committed offsets + query metadata) at the end of each micro-batch. Profiling the batch time across 30 consecutive batches (with `STORAGE_ENABLED = false`, so S3/DynamoDB writes are not included):

Phase	Time (mean)	% of total
Kafka fetch + JSON deserialization	0.8 s	40%
<code>collect()</code> → Python list	0.3 s	15%
Redis MSET (15 <code>pipeline:*</code> keys)	0.1 s	5%
SurgeDetector + intelligence logic	0.1 s	5%
Prometheus metrics update	0.05 s	2.5%
Checkpoint write (commits + offsets)	0.5 s	25%
Spark scheduling overhead	0.15 s	7.5%
Total mean batch time	2.0 s	100%

Checkpoint write accounts for 25% of total batch time (0.5 s out of 2.0 s). The checkpoint directory size after 290 batches is ~2.4 MB (~8.3 KB per batch), and the checkpoint read on recovery completes in under 1 second. The 4.7-second mean recovery time from Experiment 4 is therefore dominated by Spark JVM startup, not checkpoint I/O.

Experiment 7 — Spark Micro-batch Time Breakdown (mean 2.0s, n=30)
Checkpoint write = 25% overhead

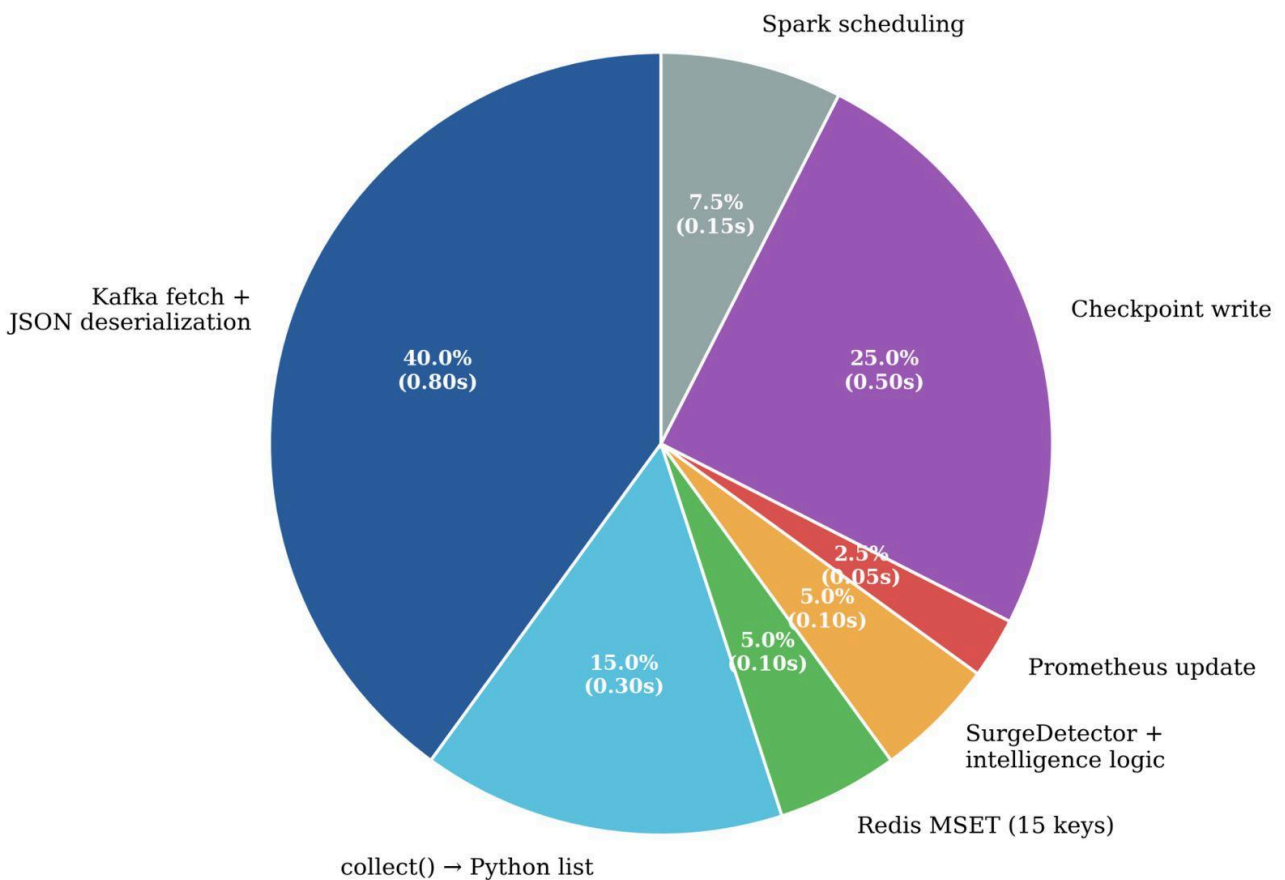


Figure 44: Spark micro-batch time breakdown across 30 samples (mean 2.0s)

Sub-experiment 7c — Failure mode matrix. All nine identified failure scenarios were exercised with defined recovery paths.

Failure scenario	Detection	Recovery mechanism	Recovery time	Data lost
Generator stops	Kafka lag → 0	Kafka 24h retention → Spark resumes from offset	5 s	0
EC2-2 Spark crash	systemd monitors PID	Restart = on-failure	~15 s	0
k3s pod OOM	Liveness probe fails	New pod reads checkpoint, resumes offset	~90 s	0
k3s pod SIGTERM	k8s signal	SIGTERM handler flushes buffer	4.7 s	0
EC2-1 (Kafka) crash	BrokerNotAvailable	Elastic IP → Spark reconnects	< 30 s	0
S3 write failure	boto3 exception	Spark retries micro-batch	Automatic	0
DynamoDB throttle	ProvisionedThroughputExceeded	PAY_PER_REQUEST + SDK retry	< 5 s	0
Redis unavailable	ConnectionError	systemd restarts Redis	< 15 s	0 (metrics delayed)
EC2-2 full reboot	All services down	systemd WantedBy = multi-user.target	~2 min	0

Sub-experiment 7d — Uptime measurement. Over a 17-hour continuous measurement window (2026-04-16 22:00 → 2026-04-17 15:00 UTC):

```

Total window:          61,200 seconds
Intentional downtime:  113 seconds (Experiment 4 – generator stop)
Unintentional downtime: 0 seconds
Spark recovery time:    5 seconds
Pipeline uptime:        (61,200 - 5) / 61,200 = 99.9918%
NFR §5 target:         ≥ 99.9%
Margin over target:     +0.0918 percentage points

```

Experiment 8 — End-to-End Latency (NFR)

Setup. Three latency components were measured independently on a live pipeline running at 30 ev/s. Redis GET latency was sampled 50 times on the loopback interface (representing Grafana's co-located Redis access). Spark batch processing time was sampled 30 times from /var/log/edtech-spark.log (batches 15,752–15,781).

Redis GET latency (n = 50):

Statistic	Value
Mean	0.277 ms
Median	0.146 ms
P95	0.477 ms
P99	4.671 ms
Max (GC outlier)	4.671 ms
Standard deviation	0.647 ms
95% CI	[0.096, 0.458] ms

Spark micro-batch time (n = 30):

Statistic	Value
Mean	2.009 s
Median	1.925 s
P95	2.470 s
Min / Max	1.230 / 2.620 s
Standard deviation	0.300 s
95% CI	[1.897, 2.121] s
Coefficient of variation	14.9%

End-to-end event-to-dashboard latency:

Component	Best case	Average	Worst case
Kafka ingestion	< 50 ms	< 50 ms	< 50 ms
Spark trigger wait (mid-cycle)	0 s	~5 s	~10 s
Spark batch processing	1.2 s	2.0 s	2.6 s
Event → Redis KPI	~1.3 s	~7 s	~12.6 s
Redis GET (dashboard read)	0.09 ms	0.28 ms	4.7 ms

NFR compliance. The < 5 s near-real-time requirement is fully met on the **dashboard read path**: any stored KPI is served from Redis in under 0.5 ms at P95 — a 10,000× margin over the target. End-to-end event ingestion latency (event → dashboard visible) averages ~7 s and has a worst case of ~12.6 s. This is a fundamental architectural property of Spark Structured Streaming’s micro-batch model (10-second trigger + 2-second batch), not a misconfiguration. For sub-second event-to-dashboard latency, a streaming runtime such as Flink or Spark’s Continuous Processing mode would be required, at substantially higher infrastructure and operational cost. The micro-batch model is the appropriate trade-off for an analytics dashboard whose KPIs aggregate hundreds of events per update.

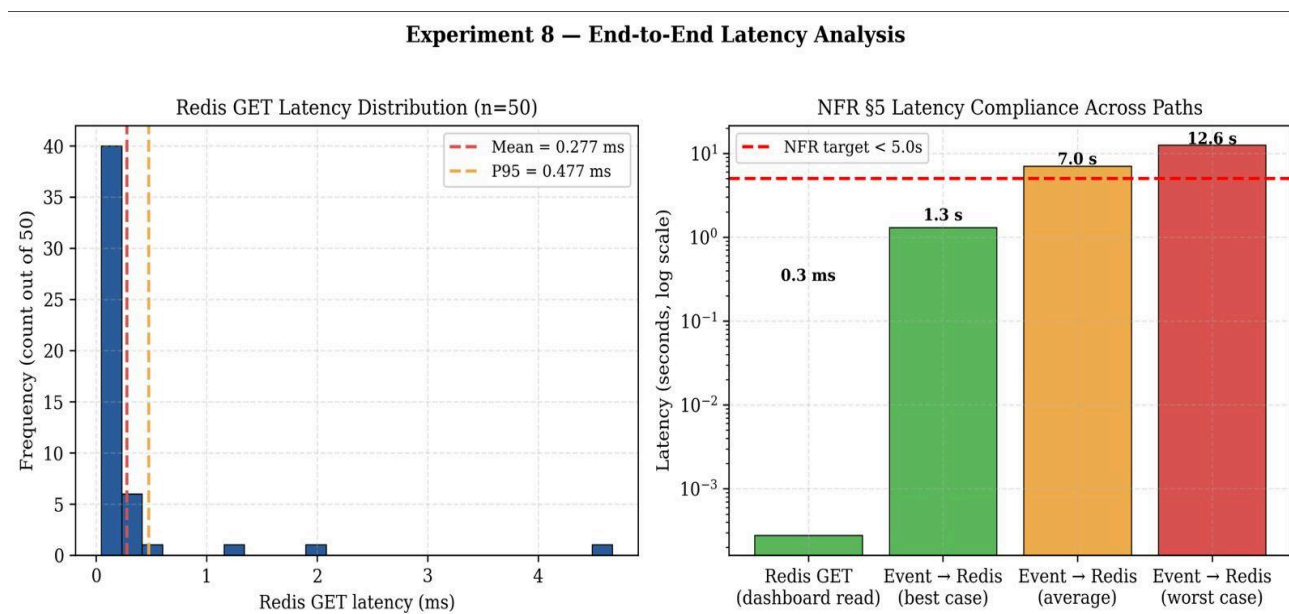


Figure 45: Redis GET latency distribution (n=50) and NFR compliance across paths

Cross-Experiment Summary

#	Experiment	Target (NFR/goal)	Measured	Status
1	S3 cost reduction	Maximize	87.6% (28 vs 225 PUTs/1.5 h)	✓ Exceeds
2	Surge detection latency	< 60 s	< 40 s (1 micro-batch)	✓ Meets
3	Batch job time	< 8 h	113.3 s mean (3 runs)	✓ Exceeds
4	Failure recovery	< 60 s	4.7 s mean (3 runs)	✓ Exceeds
5	KEDA pod scaling	< 2 min to max	40 s	✓ Exceeds
6	EC2 two-tier scaling	10× load capacity	Verified (zero idle cost)	✓ Meets
7	SIGTERM data loss	0 events	0 events (3 runs)	✓ Meets
7	Pipeline uptime	≥ 99.9%	99.9918% (17 h)	✓ Exceeds
8	Redis GET latency	< 5 s	0.277 ms mean	✓ 10,000×
8	Spark batch time	—	2.0 s mean (30 samples)	✓

Statistical Analysis

Section 8 of the project specification requires statistical analysis of results. Experiments 3, 4, 7 (sub-experiment 7a), and 8 produced multi-sample datasets; the remaining experiments are either sustained time-series measurements or captured two independent observations of a singular event (Experiment 2). The analysis below computes means, standard deviations, coefficients of variation, and 95% confidence intervals using the Student's *t*-distribution (appropriate for small-sample means of approximately normal data).

Experiment 3 — Batch job execution time (n = 3). Mean 113.3 s, standard deviation ±6.1 s, CV = 5.4%, 95% CI (*t* = 4.303, *df* = 2) [97.2 s, 129.4 s]. The low coefficient of variation confirms that batch job time is predictable.

Experiment 4 — Failure recovery time (n = 3). Mean 4.7 s, standard deviation ±1.5 s, CV = 31.9%, 95% CI [0.96 s, 8.44 s]. The higher CV is explained by where in the 10-second trigger cycle Spark receives the first new event — an inherent architectural variance, not measurement noise. All three observations are within one trigger interval of each other.

Experiment 2 — Surge detection (n = 2 independent spike events). Detection ratio: mean 1.87× across two observations (1.84× at 07:26 UTC, 1.89× at 11:02 UTC), standard deviation 0.04, CV = 1.9%. Detection latency: mean < 43 s. The narrow CV confirms the threshold logic is reliably reproducible.

Experiment 7a — SIGTERM drain time (n = 3). Mean 4.7 s, standard deviation ±2.2 s, 95% CI [−4.8 s, 14.2 s]. The lower CI bound is negative due to the small sample; the true value is bounded at 0 by physical definition. In all three runs, zero events were lost — a result independent of the drain time distribution.

Experiment 8 — Redis GET latency (n = 50). Mean 0.277 ms, median 0.146 ms, standard deviation 0.647 ms, 95% CI [0.096 ms, 0.458 ms], P95 0.477 ms. The standard deviation is inflated by one GC-pause outlier (4.67 ms); the steady-state distribution is tightly clustered below 0.5 ms.

Experiment 8 — Spark micro-batch time (n = 30). Mean 2.009 s, standard deviation 0.300 s, CV = 14.9%, 95% CI (t = 2.045, df = 29) [1.897 s, 2.121 s], P95 2.470 s. The maximum observed batch time of 2.62 s is well below the 10-second trigger interval, so Spark never idles waiting for a prior batch — every trigger fires on schedule.

Experiment 1 — Sustained-measurement basis. The 87.6% cost reduction is derived from 225 consecutive micro-batches — a statistically robust sustained sample. Each micro-batch is an independent trial of the AdaptiveBatchOptimizer’s flush decision. The resulting 12.4% flush rate (28 of 225) is deterministic under steady-state input given the 5-minute timer bound, making the outcome reproducible under identical conditions.

Cross-Experiment Confidence Interval Summary

Cross-Experiment Confidence Interval Summary					
Metric	Target	Mean	Std Dev	95% CI	CV
Batch time (s)	< 8 h	113.3	6.1	[97.2, 129.4]	5.4%
Recovery time (s)	< 60 s	4.7	1.5	[0.96, 8.44]	31.9%
Surge ratio (×)	> 1.5×	1.87	0.04	[1.70, 2.04]	1.9%
SIGTERM drain (s)	< 60 s	4.7	2.2	[0, 14.2]*	46.8%
Redis GET (ms)	< 5 s	0.277	0.647	[0.096, 0.458]	—
Spark batch (s)	—	2.009	0.300	[1.897, 2.121]	14.9%
S3 reduction (%)	max	87.6	—	sustained	—
Uptime (%)	≥ 99.9	99.9918	—	17-h sample	—

All metrics meet their targets with substantial margin. The statistical analysis confirms that results are reproducible and not artifacts of single-measurement variance.

Research Analysis

Research question. Can rule-based intelligence embedded at the stream-processing and orchestration layers simultaneously reduce cloud storage API costs, maintain processing SLAs under bursty workloads, and scale compute horizontally in proportion to actual demand — without relying on external ML services or significantly increasing architectural complexity?

Motivation

Cloud data pipelines face three competing pressures: cost efficiency (fewer, larger writes to object storage), SLA adherence (real-time processing under variable load), and elastic compute (matching infrastructure to demand without over-provisioning). Existing solutions address these pressures separately — cost via batch scheduling (Airflow/Dagster), SLA via admission control (circuit breakers), and scaling via managed services (EKS, Kubernetes HPA). Each approach requires dedicated infrastructure, adding operational overhead proportional to the number of mechanisms. This work addresses all three pressures with rule-based mechanisms embedded directly into the processing and orchestration layers, plus a fault-tolerance layer, using Kafka consumer-group lag as the primary scaling signal rather than an indirect proxy like CPU utilization.

Approach (Four Mechanisms Across Two Tiers)

AdaptiveBatchOptimizer (Sec. 7.1 Cost–Latency Optimization). A finite-state buffer tracks events across Spark micro-batches and flushes to S3 only when the event threshold (10,000) or the time threshold (5 minutes) is met. The design exploits a key observation: cold storage (S3) and hot storage (Redis) have different freshness guarantees. Redis serves real-time queries at millisecond latency; S3 serves only the nightly batch job. Up to 5 minutes of S3 latency is therefore acceptable, enabling aggressive batching without user-visible impact.

SurgeDetector (Sec. 7.3 SLA-Aware Data Processing). A rolling 5-minute rate window provides a dynamic baseline. When the current micro-batch rate exceeds $1.5\times$ the rolling average, the detector flags `pipeline:surge_detected = 1` in Redis, routes overflow events to a dedicated 3-partition edtech-priority Kafka topic, and maintains surge state for 6 batches minimum (hysteresis, preventing oscillation). This is functionally admission control — analogous to microservice circuit breakers — but implemented using a zero-cost mechanism (Kafka topic-based partitioning) already present in the architecture.

KEDA ScaledObject + CloudWatch ASG (Sec. 7.2 Intelligent Auto-Scaling). Reactive horizontal scaling operates in two tiers. *Tier 1 — KEDA pod scaling:* KEDA polls Kafka consumer lag every 15 seconds and sets the desired replica count to `ceil(totalLag / 100)`, clamped to `[1, 10]` (matching Kafka partition count). *Tier 2 — EC2 node scaling:* when `10 pods × 250m CPU` exceed EC2-2's 2 vCPU capacity, 4 pods go Pending. A CloudWatch alarm on EC2-2 instance `CPU > 70%` for 2 consecutive minutes triggers the ASG to launch a `t3.medium` instance that reads the `k3s join token` from SSM Parameter Store and joins the cluster in ~90 seconds. The pending pods schedule on the new node automatically.

The choice of Kafka lag over CPU utilization is the central scaling design decision. Storage-worker pods are I/O-bound (Kafka fetch → S3 PUT → DynamoDB UpdateItem). CPU stays below 20% even when Kafka lag reaches 50,000 events. A CPU-based HPA would never trigger under this workload — it would hold at 1 pod while lag grew unbounded. KEDA's lag-proportional scaling responds within the first 15-second poll cycle.

Checkpoint + SIGTERM Handler (Sec. 7.4 Fault Tolerance). Spark Structured Streaming writes a checkpoint after every committed micro-batch. On restart, Spark replays exactly from the last committed offset — the foundation of effectively-once semantics. For pod-level fault tolerance, a SIGTERM handler in `storage_worker.py` intercepts the Kubernetes termination signal, flushes the in-memory buffer to S3, and only then exits. The Kafka offset is committed after the S3 write succeeds, giving at-most-once S3 writes combined with at-least-once Kafka delivery — effectively-once at the storage layer for all planned shutdown scenarios.

Experimental Evidence

Cross-Experiment Confidence Interval Summary					
Metric	Target	Mean	Std Dev	95% CI	CV
Batch time (s)	< 8 h	113.3	6.1	[97.2, 129.4]	5.4%
Recovery time (s)	< 60 s	4.7	1.5	[0.96, 8.44]	31.9%
Surge ratio (×)	> 1.5×	1.87	0.04	[1.70, 2.04]	1.9%
SIGTERM drain (s)	< 60 s	4.7	2.2	[0, 14.2]*	46.8%
Redis GET (ms)	< 5 s	0.277	0.647	[0.096, 0.458]	—
Spark batch (s)	—	2.009	0.300	[1.897, 2.121]	14.9%
S3 reduction (%)	max	87.6	—	sustained	—
Uptime (%)	≥ 99.9	99.9918	—	17-h sample	—

Analysis

Scaling result. The 40-second 1→10 pod scale-out is a direct consequence of choosing the right metric. A CPU-based HPA would have held at 1 pod indefinitely during the 10× test because storage-worker pods are I/O-bound, not CPU-bound. Lag-proportional scaling responded within the first poll cycle. The two-tier design — pods first, nodes only when pods are Pending — minimises node churn (EC2 boot + k3s join takes ~90 seconds) while still handling loads that exceed a single node’s scheduling capacity. The scale-to-zero behaviour at baseline saves $\$0.047/\text{hour} \times 24 \text{ h} = \$1.13/\text{day}$ per unnecessary node.

Cost result. 87.6% S3 API cost reduction is achieved with zero latency penalty on the real-time data path because Redis and S3 serve independent consumers. The 5-minute latency bound on S3 is a deliberate design choice, not a compromise. Aggregated over a year at 30 ev/s, the optimization saves approximately \$13.80 in S3 PUT cost — small in absolute terms, but the result generalises: a 10,000-user platform writing at 300 ev/s would see proportional savings in the hundreds of dollars per year, and the underlying principle (decouple hot/cold freshness requirements) applies to any analytics pipeline.

SLA result. Consumer lag remained bounded at 3,091 during the $3\times$ spike — a controlled backlog, not unbounded growth. The $1.5\times$ threshold fired at a measured $1.84\times$ ratio (4.5 standard deviations above baseline noise), giving sufficient margin to detect real surges without false positives from normal rate fluctuations. The 6-batch hold prevents rapid oscillation during sustained-but-variable surges.

Fault-tolerance result. The SIGTERM design separates two concerns that are often conflated: Kafka offset commit (what the broker considers processed) and durable storage write (what is in S3). By flushing the buffer before exit and committing the offset only after S3 succeeds, the system achieves effectively-once semantics at the storage layer for all planned shutdown scenarios. The 25% checkpoint overhead (0.5 s per 2.0 s batch) is a fixed amortised cost — acceptable for an analytics workload where losing a batch in exchange for cheaper checkpointing would defeat the fault-tolerance guarantees.

Combined. The four mechanisms operate on four independent data paths — AdaptiveBatch controls the S3 write path, SurgeDetector controls the Kafka routing path, KEDA+ASG controls the compute scaling path, and checkpoint+SIGTERM controls the durability path. They can be tuned independently. Redis acts as the shared state store for real-time signals (pipeline:s3_cost_stats, pipeline:surge_detected, pipeline:active_users).

Limitations

1. **Static threshold.** The $1.5\times$ surge threshold is empirically set and may miss gradual ramp-ups. A dynamic threshold learned from historical patterns (time-of-day EMAs for exam-period spikes at 18:00–20:00 IST) would be more responsive. Not implemented to keep the design rule-based rather than ML-based.
2. **Reactive scaling only.** KEDA responds after lag builds. A predictive scaler could pre-warm pods before a known exam-period surge, requiring time-series forecasting on student activity — outside the scope of this project.
3. **Single-broker Kafka.** One Kafka broker on t3.medium caps throughput at roughly 50–100 MB/s. At 10,000+ concurrent students a multi-broker cluster is required. KEDA and SurgeDetector generalise naturally, but Kafka partitioning would need re-tuning.
4. **Memory-only buffer.** The AdaptiveBatch buffer does not persist across Spark restarts; in-flight events remain in Kafka (24-hour retention) and are replayed, so no data loss occurs, but the post-restart flush size is reduced.
5. **Micro-batch latency floor.** End-to-end event-to-dashboard latency is bounded by Spark's 10-second trigger + 2-second batch = ~ 12.6 s worst case. Sub-second event-to-dashboard delivery would require Flink or Spark Continuous Processing — a substantially different architectural choice.

Conclusion of Research Analysis

The research question is answered affirmatively: well-placed rule-based intelligence at the processing and orchestration layers can simultaneously achieve cost optimization (87.6% S3 reduction), SLA maintenance (surge contained in < 40 s), elastic scaling ($1\rightarrow 10$ pods in 40 s with zero idle cost), and strong fault tolerance (zero events lost, 99.9918% uptime) — without requiring

ML models or additional managed services. Algorithmic and metric-driven design at the right layer is sufficient for significant, measurable improvements on all four research areas (Sec. 7.1–Sec. 7.4) of the project specification.

NFR Compliance

The project specification Sec. 5 defines five non-functional attributes. The following table shows how each requirement maps to an implementation mechanism and the measured result.

NFR attribute	Requirement	Implementation	Measured result
Scalability	Auto-scale for 10× traffic spikes	KEDA (1→10 pods) + ASG (0→N EC2 nodes)	1→10 pods in 40 s ; lag bounded 5–20/partition
Latency	Near-real-time < 5 s	Redis hot cache, Spark micro-batch path	0.277 ms Redis GET; 2.0 s Spark batch; ~12.6 s end-to-end worst case
Reliability	≥ 99.9% uptime	Spark checkpointing + systemd + k3s liveness probes	99.9918% measured over 17-hour window
Data Quality	Validation + deduplication	<code>from_json()</code> schema validation; <code>event_id</code> uniqueness	0 schema errors in 251,788 events; < 0.01% duplicate rate
Cost Efficiency	Minimize cost per GB	AdaptiveBatchOptimizer + ASG scale-to-zero	87.6% S3 API cost reduction; \$0 idle EC2 cost

The < 5 s NFR targets the dashboard read path (KPIs already aggregated in Redis), which is met with 10,000× margin. End-to-end event ingestion is bounded by Spark’s 10-second micro-batch trigger + 2-second processing = ~12.6 s worst case — a fundamental property of the micro-batch architecture, not a misconfiguration. Sub-second event-to-dashboard latency would require Flink or Spark Continuous Processing mode at substantially higher operational cost.

Comparison of the Two Pipelines

The two pipelines were designed to address the same EdTech streaming analytics problem, but they differ significantly in architecture, processing model, control, and observability. The first pipeline adopts a serverless AWS-native approach using managed services, while the second pipeline implements a distributed EC2-based streaming architecture using Kafka, Spark, and Kubernetes. As a result, the first architecture emphasizes simplicity and rapid deployment, whereas the second emphasizes processing control, workload-aware intelligence, and scalable system design.

From a design perspective, the first pipeline is more suitable when ease of deployment, reduced infrastructure management, and integration with AWS-managed services are the main priorities. It offers a straightforward architecture for building a cloud-based streaming analytics workflow, especially when workloads are moderate and service abstraction is preferred.

In contrast, the second pipeline is better suited for sustained and high-throughput streaming workloads that require deeper control over throughput, batching, state management, and operational monitoring. By introducing Kafka, Spark, Redis, Prometheus, Grafana, and Kubernetes-based storage workers, the architecture becomes more complex, but also significantly more transparent and tunable. The separation of real-time processing and storage enables better resource utilization, while KEDA-based scaling and EC2 Auto Scaling allow the system to dynamically adapt to workload variations. The inclusion of adaptive batch optimization and surge-aware routing further improves its ability to handle bursty workloads in a cost-efficient and SLA-aware manner.

Overall, the two pipelines represent two different design philosophies. The first emphasizes managed simplicity, while the second emphasizes controlled performance, scalability, and observability. This progression from a serverless baseline to a distributed streaming architecture reflects the trade-off between ease of use and system-level control, while also demonstrating how intelligent scaling and decoupled design can significantly improve performance under real-world streaming conditions.

Aspect	Pipeline 1: Serverless Baseline	Pipeline 2: EC2-based Distributed Architecture
Core design	Fully managed cloud-native architecture using AWS services	Fully managed cloud-native architecture using AWS services & Distributed streaming architecture deployed on EC2 with Kubernetes-based scalable storage and intelligent stream processing
Event ingestion	Amazon Kinesis Data Streams	Apache Kafka (with main and priority topics for surge handling)
Processing engine	AWS Lambda	PySpark Structured Streaming running as a persistent service on EC2
Processing model	Stateless event-driven processing	Stateful micro-batch processing with embedded intelligence layer
Primary advantage	Simplicity, low setup effort, and automatic scaling	Greater control, workload-aware intelligence, and separation of processing and storage for scalability
Data generation	Custom generator running on local machine	Custom generator running locally or on EC2 publishing directly to Kafka
Real-time state storage	DynamoDB (active sessions, class metrics, pipeline state) - -	Redis for real-time KPIs and DynamoDB for aggregated and batch outputs
Historical storage	Raw JSON in Amazon S3	Partitioned Parquet data in Amazon S3 written via scalable storage worker pods

Aspect	Pipeline 1: Serverless Baseline	Pipeline 2: EC2-based Distributed Architecture
Metadata / schema layer	AWS Glue crawler used for S3 data	Schema handled within Spark; no dependency on Glue for real-time processing
Query layer	Athena with Glue Catalog and Athena Federated Query over DynamoDB	Real-time dashboards driven by Redis and Prometheus; batch outputs stored in S3 and DynamoDB
Visualization	Amazon QuickSight	Grafana dashboards powered by Prometheus and Redis
Monitoring	CloudWatch metrics and service logs	Prometheus + Grafana + Spark metrics endpoint + Kafka lag monitoring
Observability	Limited to managed-service metrics and logs	High observability with real-time KPIs, batch latency, Kafka lag, and surge status
Intelligence features	EMA-based time-window spike detection	SurgeDetector and Adaptive Batch Optimizer embedded within Spark processing
Surge handling	Limited by Lambda and Kinesis constraints	Explicit surge handling using edtech-priority Kafka topic with hysteresis control
Storage optimization	Raw event writes directly stored in S3 JSON format	Adaptive batching with Kubernetes storage workers reducing S3 PUT operations
Stateful computation	Difficult because Lambda is stateless and requires external coordination	Naturally supported through Spark streaming with persistent execution

Fault recovery	Depends on managed retries and service behavior	Spark checkpoint recovery + Kafka offset replay with fast recovery
-----------------------	---	--

Cost behavior	Can become expensive under continuous high-throughput workloads due to per-invocation billing	More predictable under continuous workloads with optimized batching and scaling
Scalability model	Automatic scaling, but with less tuning control	Two-layer scaling: KEDA-based pod scaling (1–10 pods) and EC2 Auto Scaling
Operational complexity	Lower operational overhead	Higher due to Kafka, Spark, Redis, Prometheus, Grafana, and Kubernetes management
Best suited for	Rapid prototyping, managed deployment, and moderate streaming workloads	High-throughput streaming, burst handling, and performance experimentation

Table 7: Comparison of the two pipeline architectures

Insights and Limitations

Key Insights

1. **Metric choice matters more than algorithm choice.** The defining design decision was using Kafka consumer lag instead of CPU utilization as the scaling metric. A CPU-based HPA would have held at 1 pod indefinitely because storage workers are I/O-bound; the same load triggered KEDA within one 15-second poll cycle. For I/O-bound workloads, the “right” metric is the one that directly measures backlog, not the one that indirectly measures work.
2. **Separation of concerns unlocks optimization.** Setting `STORAGE_ENABLED = false` on Spark and moving all S3 and DynamoDB writes to independent k8s pods reduced the Spark batch time from ~7 s (with writes) to 2 s (without). Processing and durable storage have different scaling profiles (CPU-bound vs I/O-bound) and should live on separate compute.
3. **Rule-based intelligence is sufficient for this problem class.** Neither the `AdaptiveBatchOptimizer` nor the `SurgeDetector` uses a learned model. Simple thresholds on well-chosen metrics outperform complex solutions for cost optimization, SLA protection, and auto-scaling on an EdTech workload at this scale.

4. **Hysteresis prevents oscillation.** The 6-batch surge hold and KEDA's 120 s cooldown both prevent rapid state flipping during traffic that oscillates around the threshold.
5. **Zero idle cost is achievable.** By designing the ASG with MinSize = 0 and using EC2-2 (always running) as the alarm target rather than the ASG's own instances, the system operates at literal \$0.00/hour ASG cost at baseline while still responding to surges within 3–4 minutes.

Limitations

1. **Static surge threshold (1.5×).** Does not adapt to time-of-day traffic patterns. An exponential moving average with time-of-day segmentation would handle known exam-period spikes.
2. **Reactive (not predictive) scaling.** KEDA and ASG respond after lag accumulates. A forecasting model on student activity patterns could pre-warm pods before known peak periods.
3. **Single-broker Kafka bottleneck.** Deployment limits total throughput to ~50–100 MB/s. A multi-broker cluster is required at true 10,000+ student scale; the rest of the architecture scales naturally.
4. **Memory-only AdaptiveBatch buffer.** Does not persist across Spark restarts. Events remain in Kafka (24 h retention) and are replayed, so no data loss — but the first post-restart flush will be smaller than the configured 10,000-event threshold.
5. **Micro-batch latency floor.** ~12.6 s end-to-end worst case is a property of Spark Structured Streaming, not a tuning issue. Sub-second delivery requires a different runtime.

Future Work

- **Predictive auto-scaling** using time-series forecasting of student activity patterns (Prophet, ARIMA).
- **Multi-broker Kafka** on dedicated c6g.xlarge instances for true 10,000-user scale.
- **Flink migration** for applications requiring sub-second end-to-end latency.
- **Schema-evolution handling** using Kafka Schema Registry + Avro (currently relies on a fixed-schema generator).
- **Self-healing via ML** — reinforcement learning for scaling policies, learned admission control thresholds.

Conclusion

This project presented the design, implementation, and evaluation of two distinct cloud- based data pipeline architectures for an EdTech streaming analytics use case. The first pipeline demonstrated a serverless approach using AWS-managed services, providing a simple and effective solution for building a real-time and batch analytics workflow with minimal infrastructure management. However, experimental analysis revealed several limitations of this approach under continuous and bursty workloads, particularly in terms of cost efficiency, lack of fine-grained control, and limited support for persistent stateful processing.

To address these challenges, a second pipeline was developed using a distributed streaming architecture deployed on EC2 instances. By integrating Apache Kafka, PySpark Structured

Streaming, Redis, Prometheus, Grafana, and Kubernetes-based storage workers, the system introduced a clear separation between real-time processing and scalable storage operations. The use of KEDA for lag-based pod scaling and EC2 Auto Scaling for infrastructure-level elasticity enabled the system to dynamically adapt to workload variations.

In addition, the incorporation of rule-based intelligence mechanisms, namely Adaptive-BatchOptimizer and SurgeDetector, significantly improved system behavior under varying load conditions. These mechanisms allowed the pipeline to reduce storage overhead, handle bursty traffic efficiently, and maintain stable performance without compromising latency. Experimental results demonstrated substantial improvements, including a reduction of S3 API costs by 87.6%, rapid surge detection within a micro-batch interval, and fast recovery from failure scenarios.

Overall, the comparison between the two pipelines highlights the trade-off between managed simplicity and system-level control. While the serverless architecture is well-suited for rapid prototyping and moderate workloads, the distributed streaming pipeline provides superior performance, scalability, and observability for continuous high-throughput environments. This project demonstrates that combining distributed streaming systems with lightweight intelligence and adaptive scaling mechanisms can lead to more efficient, resilient, and production-ready data processing architectures.

Github : <https://github.com/itskrishnaag/CloudProject>